



Order Book Analyzer – Status Report 3

FINTECH 512 - Software Engineering for FinTech

Full team of five members for the group project (section 1)



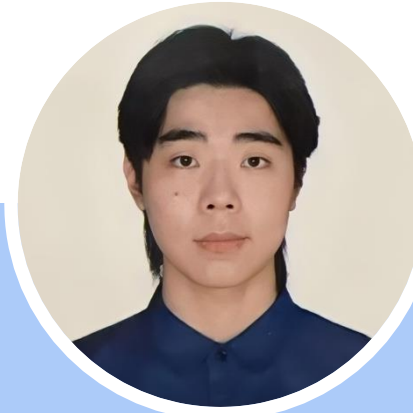
Rebeca Hechtman
NetId: rh398



Mike Lin
NetId: hl561



Mory Shi
NetId: cs836



Loki Luo
NetId: lz294



Alan Song
NetId: ys501

Agenda

- 01.** Project Ideation
- 02.** GitLab Repository and Issues
- 03.** Updated Design
- 04.** Status, Review and Planning

Project Ideation

Project's 'elevator pitch'



Bloomberg of Prediction Markets



Prediction markets, where people bet real money on the outcomes of future events (like whether Bitcoin will hit \$110,000 this year), are exploding in popularity. But making sense of all that betting activity can be frustrating. The raw data behind these markets is messy, disorganized, and hard to easily analyze.

Order Book Analyzer cuts through that noise. Think of us as the Bloomberg for prediction markets. We take massive amounts of complicated, second-by-second betting data and translate it into clear, powerful dashboards and insights. Instead of wrestling with messy data, our platform gives you the exact trends and historical information you need to see exactly where the smart money is going.

Project Scope Disclaimer: This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1). Future market expansions remain under evaluation based on technical complexity and timeline constraints.

The Lean Canvas

Designed for:

Order Book Analyzer

Designed by:

Team Hotel

Date:

03/16/2026

Version:

2.0

Problem  1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution  A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop.  High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage  1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments  - Mid and small size Hedge Funds - Sophisticated algo traders who wants to trade in prediction market
Existing Alternatives  - Eagle Alpha - Dune Analytics - DIY engineering - Basic Polymarket Dashboards	Key Metrics  Growth: # of users - Retention - API call volumes - Daily Active User System Health: - Data Quality Score	High-Level Concept  - Yahoo Finance for Prediction Market - Bloomberg for Prediction Market	Channels  - Social Media (LinkedIn, X, Reddit, Red Note) - Organic Growth - Search Engine - Ads	Early Adopters  Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.
Cost Structure  Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.		Revenue Streams  Revenue Model: Freemium <u>Freemium Model:</u> Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. <u>Key Account (KA) Annual Licenses:</u> High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. <u>Value-Added Services (Overage):</u> Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.		

Changes we've made since the project started

The Lean Canvas		Designed for: Order Book Analyzer	Designed by: Team Hotel	Date: 03/16/2026	Version: 2.0
Problem  1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution  A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop.  High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage  1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments  1. Mid and small size Hedge Funds 2. Sophisticated algo traders who wants to trade in prediction market	
Existing Alternatives  1. Eagle Alpha 2. Dune Analytics 3. DIY engineering 4. Basic Polymarket Dashboards	Key Metrics  Growth: 1. # of users 2. Retention 3. API call volumes 4. Daily Active User System Health: 1. Data Quality Score	High-Level Concept  1. Yahoo Finance for Prediction Market 2. Bloomberg for Prediction Market	Channels  1. Social Media (LinkedIn, X, Reddit, Red Note) 2. Organic Growth 3. Search Engine 4. Ads	Early Adopters  Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.	
Cost Structure  Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.		Revenue Streams  Revenue Model: Freemium <u>Freemium Model:</u> Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. <u>Key Account (KA) Annual Licenses:</u> High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. <u>Value-Added Services (Overage):</u> Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.			

Lean Canvas is adapted from The Business Model Canvas (www.businessmodelgeneration.com/canvas). PowerPoint Implementation by: Neos Chronos Limited (<https://neoschronos.com>). License: CC BY-SA 3.0

Scope Restriction:

- This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1).
- Future market expansions remain under evaluation based on technical complexity and timeline constraints.

Prototype Screens

1- Home Page

for unauthenticated users + User authentication page

OBAnalyzer

Create AccountSign In

Order Book
Analyzer
for Prediction Markets

Capture, store and replay real-time market data from Polymarket and BTC/USD. Built for backtesting and algorithmic strategy development.

Create Free Account

Real-time

WebSocket ingestion

Replay

Tick-level playback

Depth

1s & 5s intervals

Welcome back

Sign in to access your analytics.

Continue with Google

Continue with GitHub

OR

EMAIL

trader@example.com

PASSWORD

.....

☒ Remember meForgot password?

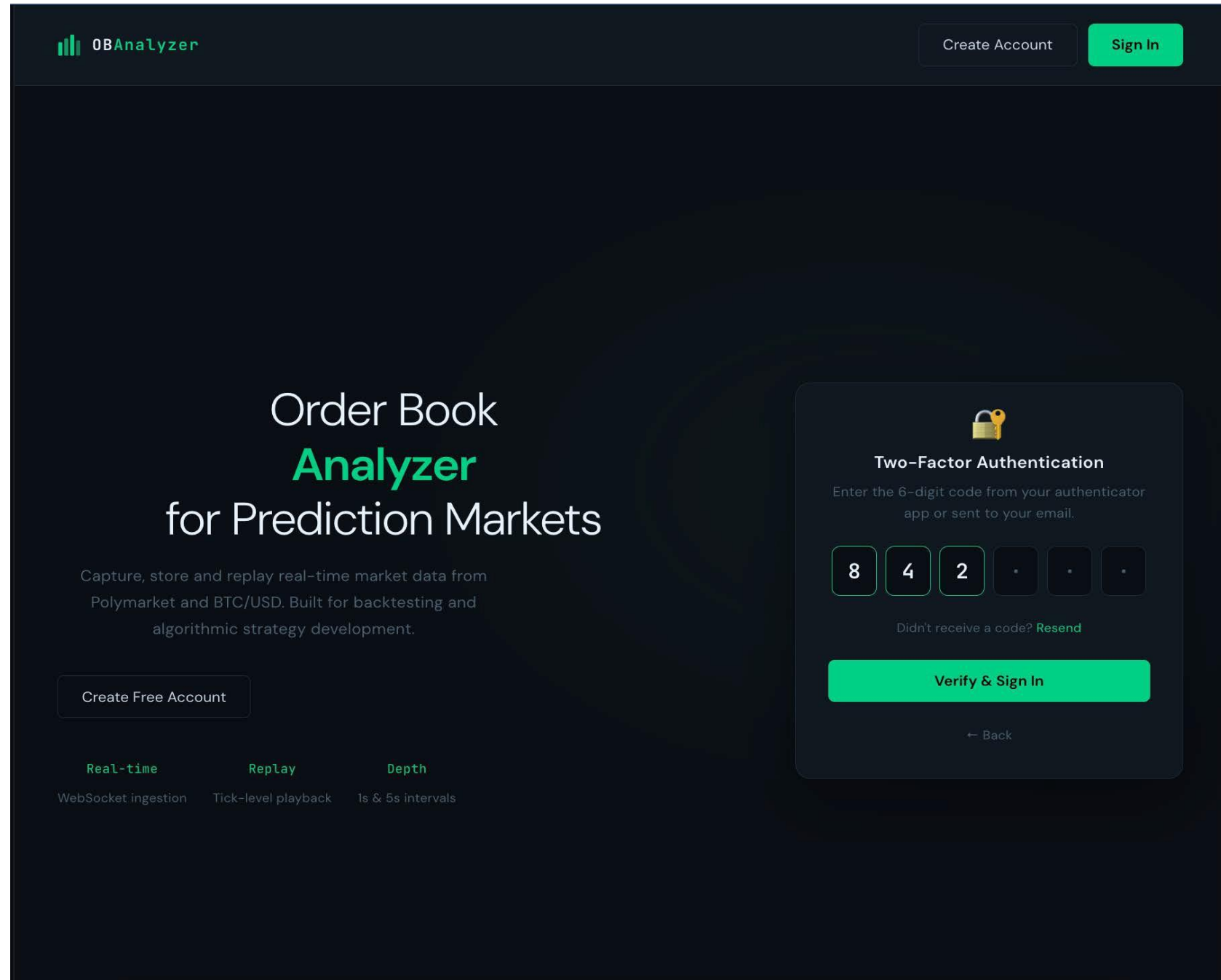
Sign In

No account? [Create one free](#)

256-bit SSL · SOC 2 Type II

Prototype Screens

1b- User Authentication



Prototype Screens

2- New user registration page

OBAnalyzer

Already have an account? [Sign In](#)

FREE ACCESS

Start analyzing
prediction markets

Get instant access to BTC order book replays, binary option depth charts, and historical snapshot data.

 **Dashboard**
Visualize BTC prices & 15-min binary options

 **Replay Engine**
Full order book playback at any timestamp

 **Depth Charts**
Cumulative depth at tick, 1s and 5s

Create Account

Set up your profile in seconds

FIRST NAME

LAST NAME

Jane

Doe

EMAIL

trader@example.com

USERNAME

@handle

PASSWORD

Min. 8 characters

CONFIRM PASSWORD

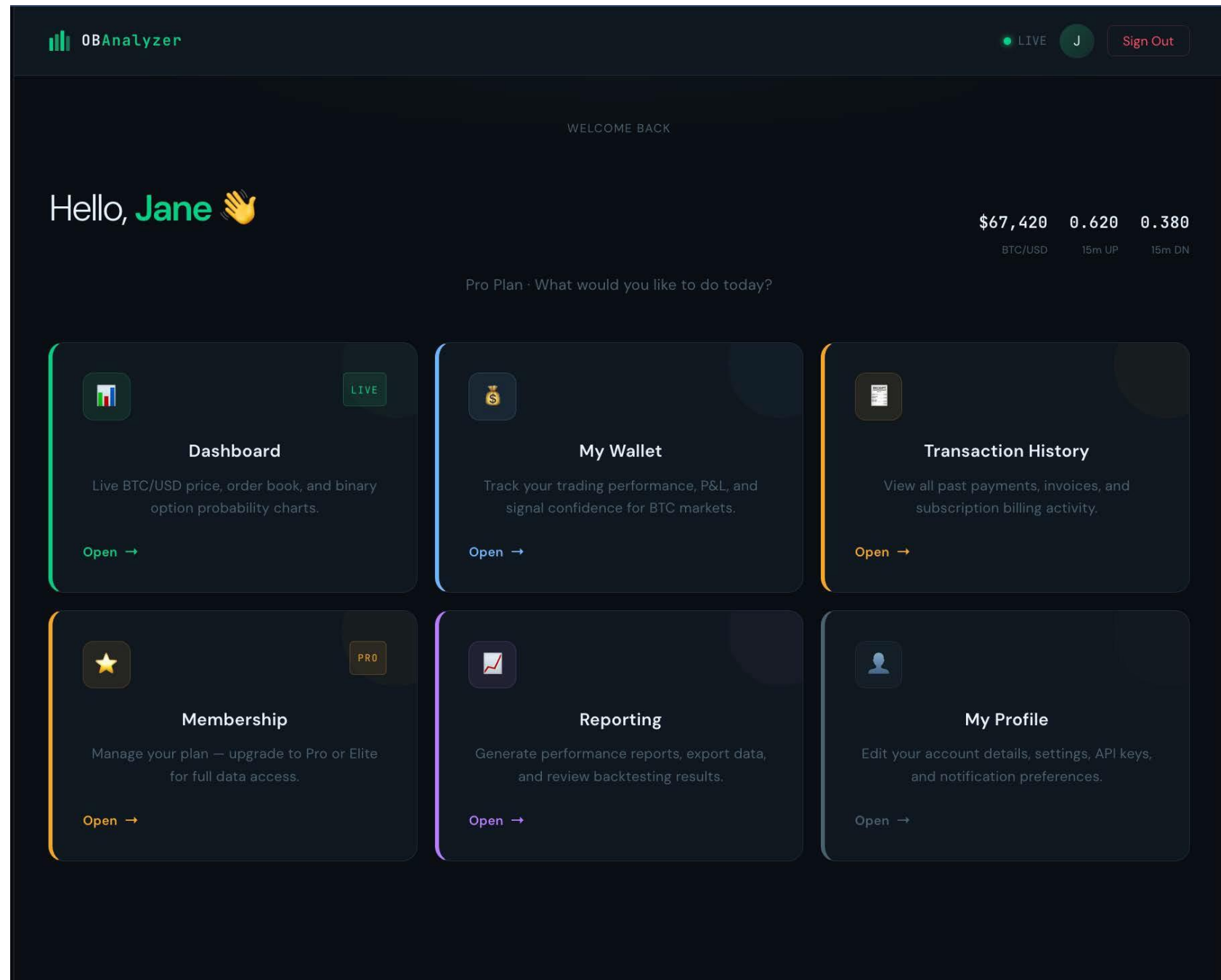
Repeat password

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

Create Account →

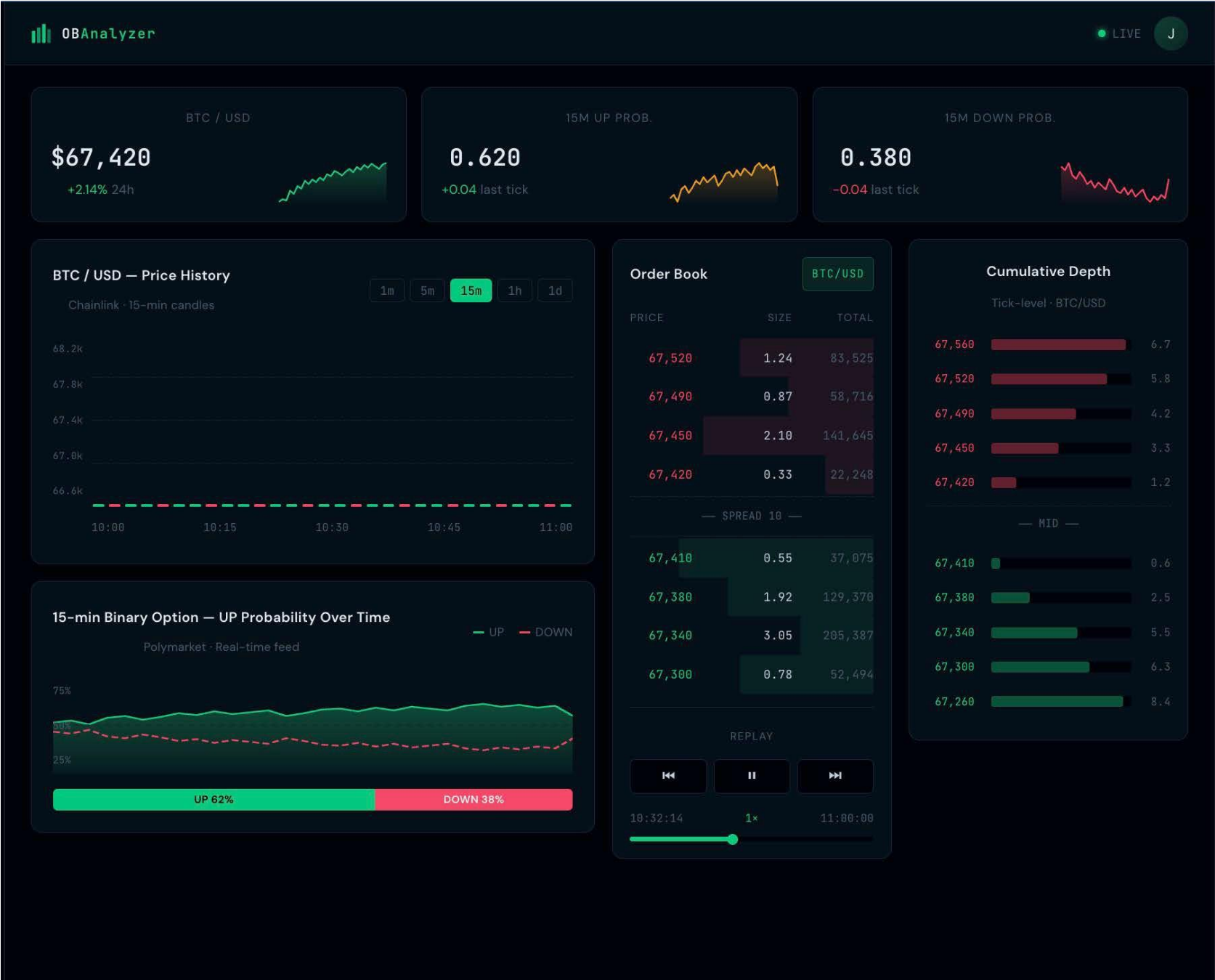
Prototype Screens

3- Authenticated home page for regular users



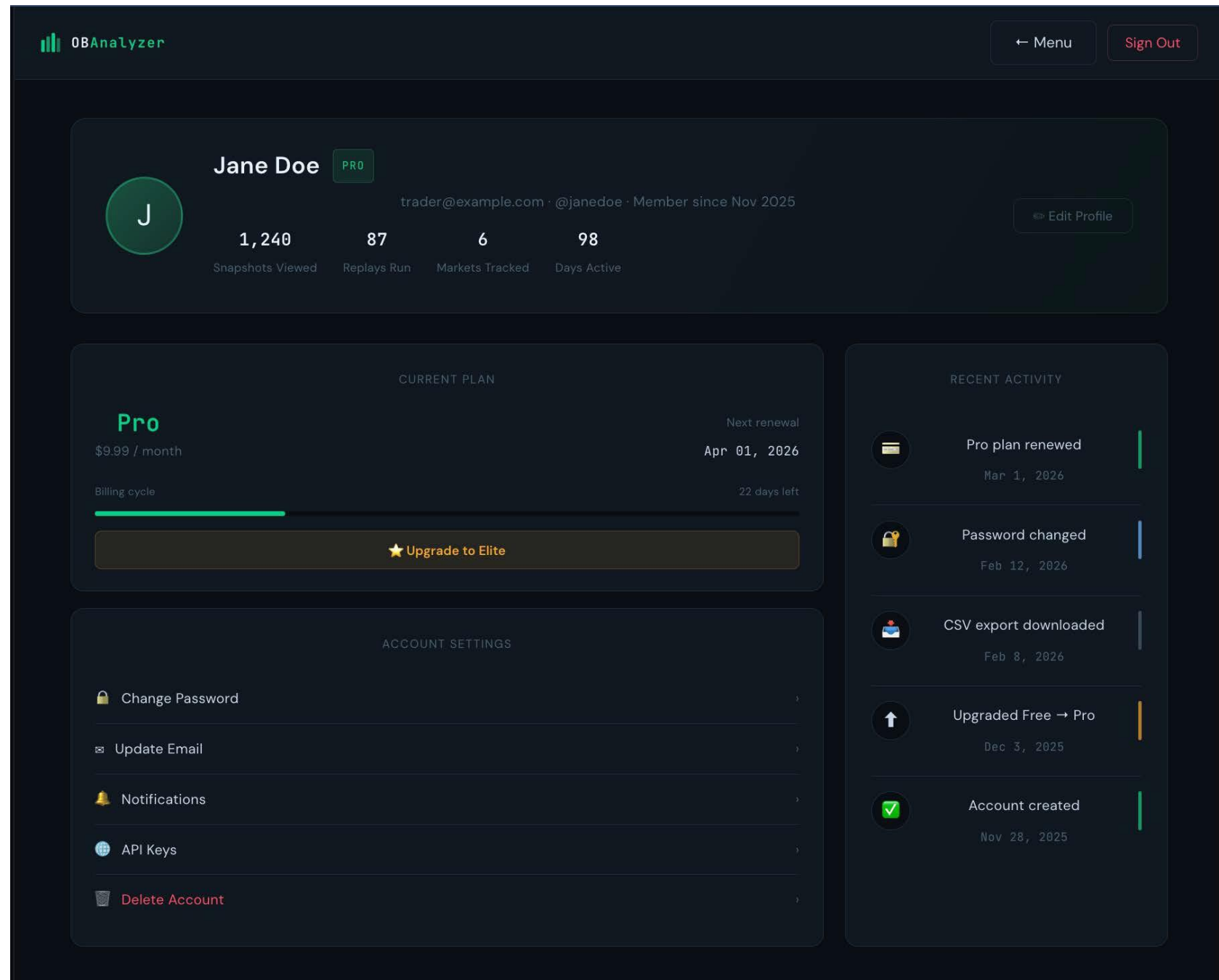
Prototype Screens

4- Dashboard of relevant information



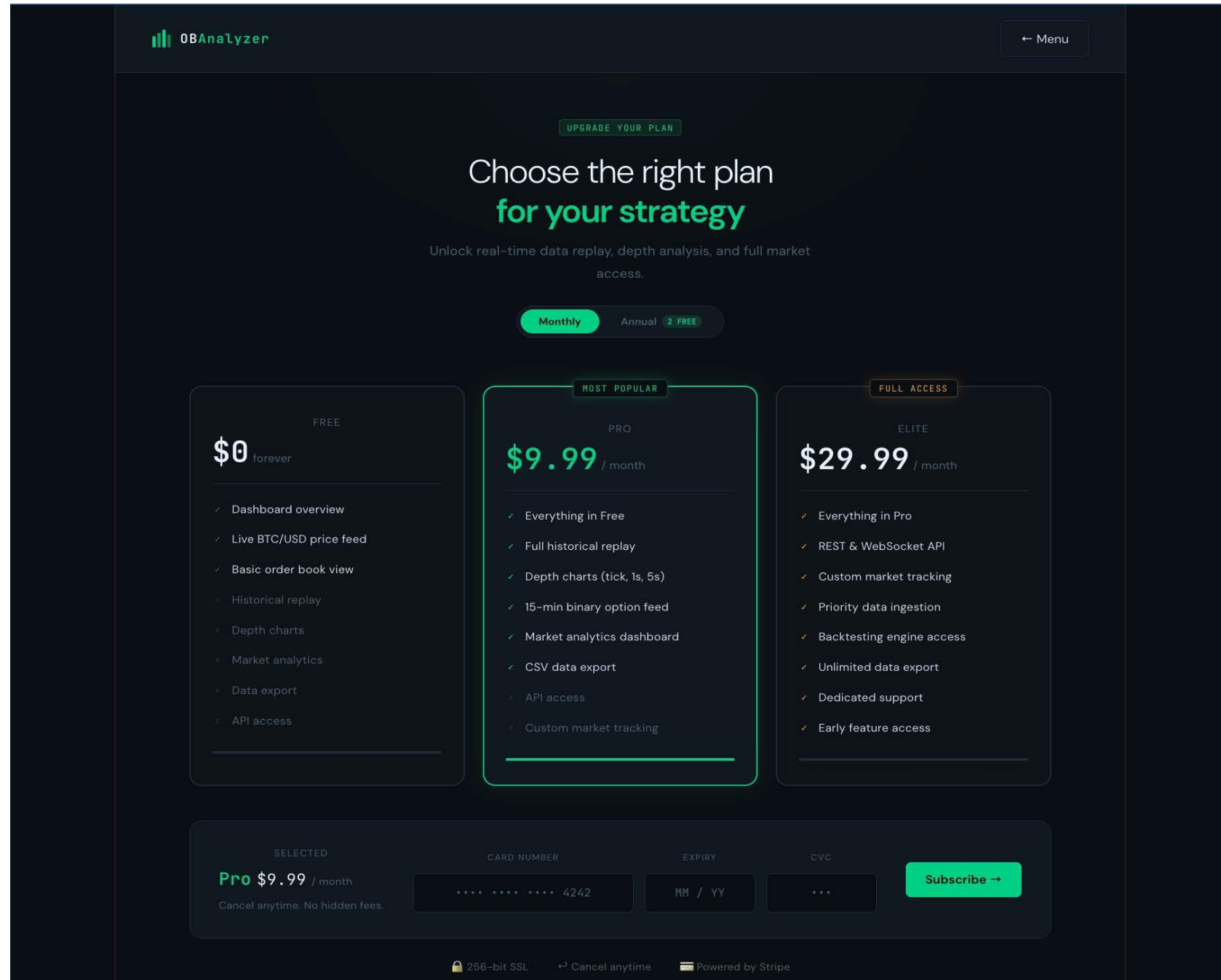
Prototype Screens

5- User information



Prototype Screens

6- Buy product / service



Prototype Screens

7- History screen to show past transactions

OBAnalyzer

Manage Plan← Menu

ACCOUNT

Transaction History

All billing activity for your account.

CURRENT PLAN

Pro

Active since Dec 2025

TOTAL BILLED

\$662.00

Lifetime spend

TRANSACTIONS

10

All time

NEXT RENEWAL

Apr 01, 2026

\$29.00 / month

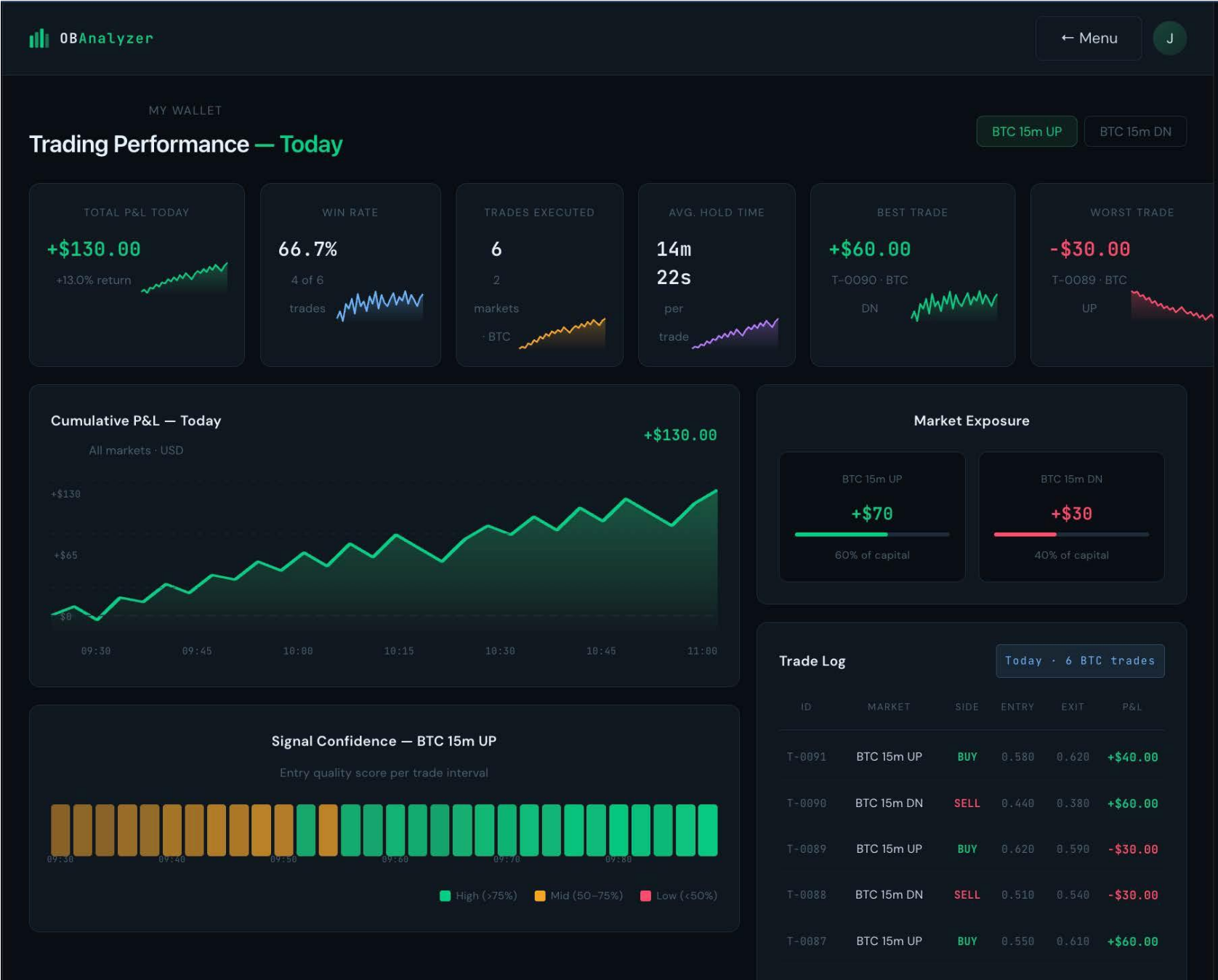
Invoices & Payments

AllPaidRefunded

TRANSACTION ID	DATE	DESCRIPTION	AMOUNT	PAYMENT METHOD	STATUS
TXN-00142	Mar 01, 2026	Pro Plan — Monthly	\$29.00	Visa •• 4242	Paid
TXN-00141	Feb 01, 2026	Pro Plan — Monthly	\$29.00	Visa •• 4242	Paid
TXN-00140	Jan 01, 2026	Pro Plan — Monthly	\$29.00	Visa •• 4242	Paid
TXN-00139	Dec 03, 2025	Upgrade: Free → Pro	\$29.00	Visa •• 4242	Paid
TXN-00138	Nov 15, 2025	Elite Plan — Monthly	\$99.00	Visa •• 8891	Refunded
TXN-00137	Nov 01, 2025	Elite Plan — Monthly	\$99.00	Visa •• 8891	Paid
TXN-00136	Oct 01, 2025	Elite Plan — Monthly	\$99.00	Visa •• 8891	Paid

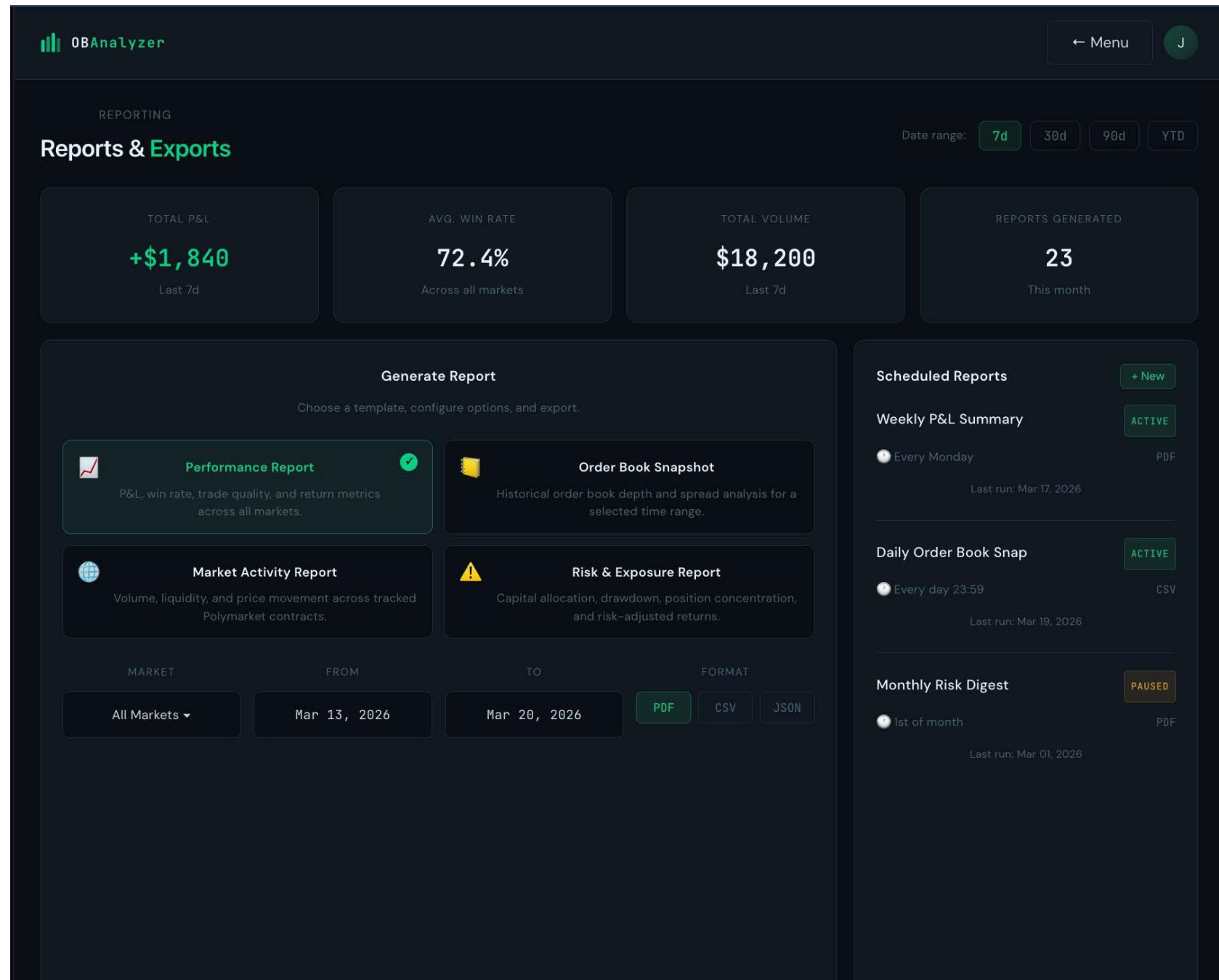
Prototype Screens

8- Analysis page of the user's current data



Prototype Screens

9 - Reporting functionality



Charting – v1 Ideation

Bitcoin Prediction Market — 15-min Contracts Microstructure Dashboard

Timeframe: 15-min · Venue: Prediction Market · Update: {YYYY-MM-DD HH:MM UTC} · Notional basis: USD

Spread (bps) · P50 / P95

{3.2} / {12.8}{+0.6} bps

Slippage (USD) · \$10k / \$50k / \$100k

{1.8} / {7.9} / {15.6}

Realized Vol (EWMA) · 15-min

{42.1}%{-4.2} pp

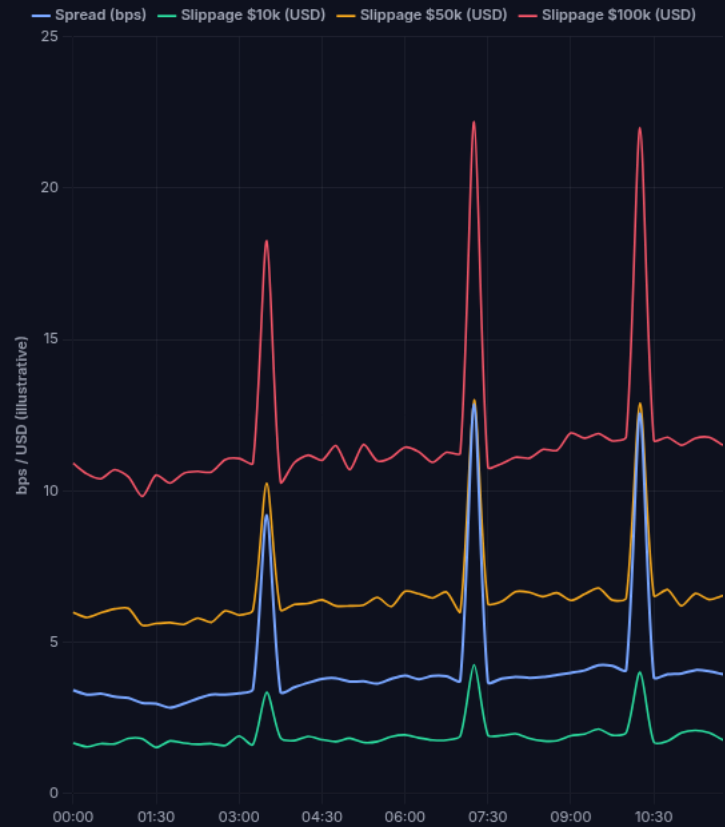
VPIN · Toxicity Alert

{0.62} HIGH(threshold: 0.70)

↔ Bid-Ask Spread Width & Slippage Curves

Spread spikes often precede news or liquidity withdrawal. Slippage curves approximate execution cost for market orders.

Microstructure Costs (15-min)



↗ Realized Volatility vs. Price Drift

Dual-axis view: BTC price (left) and EWMA realized volatility (right). Vol often decays as resolution approaches.

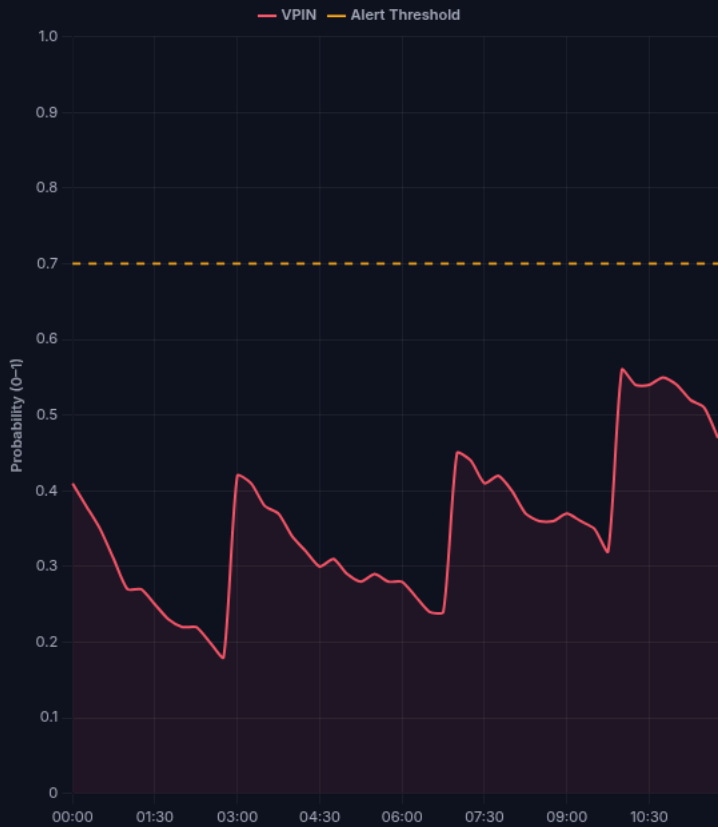
Price Drift vs Volatility Regime



⚠ Order Flow Toxicity (VPIN)

Volume-synchronized probability of informed trading. Elevated VPIN may trigger market-maker liquidity pullbacks.

Informed Trading Risk (Volume Buckets)



Method notes: Spread = (Ask - Bid) / Mid (bps). Slippage curves are theoretical cost estimates (illustrative) calibrated to instantaneous depth. Realized vol shown as EWMA of 1-min returns annualized (placeholder). VPIN computed on volume buckets (placeholder). Replace placeholders with your venue's tick/orderbook feed.

GitLab Repository and Issues

Team Project GitLab Repository

https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#

The screenshot shows a web browser displaying the GitLab interface for a repository named 'pred_market_data_platform' under the 'team-hotel' namespace. The browser's address bar shows the URL 'gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#'. The page layout includes a left sidebar with navigation options like 'Project', 'Pinned', 'Issues', 'Merge requests', 'Manage', 'Plan', 'Code', 'Build', 'Secure', 'Deploy', 'Operate', 'Monitor', 'Analyze', and 'Settings'. The main content area displays the repository's name, a message stating 'The repository for this project is empty', and sections for 'Command line instructions', 'Configure your Git identity' (with 'Local' and 'Global' tabs), 'Git local setup', 'Add files', and 'Create a new repository'. The right sidebar contains 'Project information', 'Invite your team', and a list of actions like 'Upload File', 'New file', 'Add README', 'Add LICENSE', 'Add CHANGELOG', 'Add CONTRIBUTING', 'Set up CI/CD', 'Add Wiki', and 'Configure Integrations'. The page is created on March 06, 2026.

team-hotel / pred_market_data_platform

Search or go to...

pred_market_data_platform

The repository for this project is empty
To get started, clone the repository or upload some files.

Command line instructions
You can also upload existing files from your computer using the instructions below.

Configure your Git identity
Get started with Git and learn how to configure it.

Local Global

Git local setup
Configure your Git identity locally to use it only for this project:

```
git config --local user.name "rebecca.hechtman@duke.edu"
git config --local user.email "rh398@duke.edu"
```

Add files
Push files to this repository using SSH or HTTPS. If you're unsure, we recommend SSH.

SSH HTTPS
How to use SSH keys?

Create a new repository

```
git clone git@gitlab.oit.duke.edu:team-hotel/pred_market_data_platform.git
```

Project information
Invite your team
Add members to this project and start collaborating with your team.
Invite members

Upload File
+ New file
+ Add README
+ Add LICENSE
+ Add CHANGELOG
+ Add CONTRIBUTING
+ Set up CI/CD
+ Add Wiki
+ Configure Integrations

Created on
March 06, 2026

Legend:

- Done
- In progress
- To be done

Project Requirements – Organization and Ideation

1. Organization

1. W1: List the requirements
2. W2: Refine the requirements

2. Gitlab

1. W1: Create Gitlab
2. W2: Update the issues (+ improve wording)
3. W2: Understand Gitlab SAST Tooling

3. System Architecture diagrams (C4)

1. W1: Create System Architecture
2. W2: Create using C4 model
3. W3 - Refine diagrams - Mike

4. Virtual Machines (current version of site running on a Duke VCM)

1. W1: Apply VCM
2. Share the VCM port with the group
3. Deploy the app (Gitlab -> virtual machine)
4. Automatically starting

5. Security Analysis - DFD threat model

Create a Data Flow Diagram (DFD) for your BigBucks system, identify assets, actors, and threats, and perform a STRIDE analysis of DFD

1. W2: 1st overview draft - Loki
2. W3: Refinement and detailing - Loki

6. Users and Administrative Functionality:

1. W1: initial user's type ideation
2. W2: Admin's ideation
3. Detailing of user
4. Detailing of admin

7. Data service

1. W1: data design ideation
2. W1: data flow ideation (Polymarket API)
3. W2: testing ideation
4. W4: API to connect full stack with data service – Mike/ Mory

8. Full stack

1. W2-W3: Data design of user ideation – Alan
2. W3-W4: Data design of admin ideation - Alan
3. W2: testing ideation

9. Charting (3 or more + custom interaction)

1. W3: Ideation + paper prototype - Mory
2. Refine + Include custom interaction
3. Document with definitions

10. Paper prototypes

1. W1 – W3: User Interface – Rebeca
 1. Home page for unauthenticated users + User authentication page
 2. New user registration page
 3. "Authenticated" home page for regular users
 4. Dashboard of relevant information
 5. "Analysis" page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service
 6. Analysis page of the user's current data
 7. History screen to show past transactions
 8. Reporting functionality
2. Tabs for admin functionality: View all users, Disable user accounts, Reset user passwords, View 'system' transactions, Perform some sort of analysis across all accounts

Project Requirements – GitLab and Coding

Legend:

- Done
- In progress
- To be done

11. Organization

1. W2: Setup the overall project repo structure – Rebeca
2. W1-W6: Keep GitLab issues up-to-date - Alan
3. W3: Talk about merge requests - All

12. Full Stack

1. Front-end design
 1. W3-W4: CSS for styling – Alan and Loki
 2. W3-W5: HTML for all the page prototypes (for the pages that need data, use dummy data) – Alan and Loki
 3. HTML tabs for admin functionality
2. Back-end design:
 1. JavaScript to connect front end with the API
 2. Data base with user roles and permissions
 3. OAuth2 Authentication Integration
3. Database design:
 1. for user (PostgreSQL)
4. Tests suite (including DFD created)
 1. Unit tests
5. Logging and Monitoring
 1. Understand
 2. Design
 3. Implement

13. CI/CD – YAML file

1. Build release software on every commit
2. Runs test suite on every commit
3. Runs Gitlab SAST Tooling
4. Automatically deploy your software to your VCM

14. Data Service Design

1. W2: API to connect data service with Polymarket (Data Integration using a REST/JSON web API)
2. W2: Message queue
3. W2: Database (PostgreSQL) implementation
4. W4: API to connect full stack with data service – Mike
5. W3-W5: Tests suite; also use DFD created – Mike

15. Containerize Application

1. Create docker files to create an image of the application
2. Create a docker compose file to deploy and execute image

16. Security Analysis – run tests against the software

1. Run Semgrep analyzer against your code base
2. Runs OWASP Zed Attack Proxy

Project Requirements – Detail Front End

Legend:

- Done
- In progress
- To be done

1. CSS for styling
2. HTML for all the page prototypes (for the pages that need data, using dummy data for now)
 1. Home page for unauthenticated users
 2. User authentication page
 3. New user registration page
 4. "Authenticated" home page for regular users. Dashboard of relevant information.
 5. "Analysis" page that allows the user to make an informed decision to buy/sell a particular product, item, and or service + Allow the user to buy/sell a product, item, or service
 6. Analysis page of the user's current data
 7. History screen to show past transactions
 8. Reporting functionality
3. HTML tabs for admin functionality:
 1. View all users, Disable user accounts
 2. Reset user passwords
 3. View 'system' transactions
 4. Perform some sort of analysis across all accounts

Project Issues

(updated on March 27)

Link to the issue page: https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform/-/issues?sort=created_asc&state=opened&last_page_size=20&page_before=eyJjcmVhdGVkX2F0IjoiMjAyNi0wMy0xOSAxMDozNTozMS41MDU4MzcxMDAgLTA0MDAiLCJpZCI6IjQ1NTQyIn0

 Create using C4 model team-hotel/pred_market_data_platform#30 · created 1 week ago by yinglun.song@duke.edu Done	 updated 10 minutes ago
 Refine diagrams team-hotel/pred_market_data_platform#31 · created 1 week ago by yinglun.song@duke.edu Done	 updated 10 minutes ago
 Virtual Machines (current version of site running on a Duke VCM) team-hotel/pred_market_data_platform#32 · created 1 week ago by yinglun.song@duke.edu Current in Progress	 updated 9 minutes ago
 Creation of VCM team-hotel/pred_market_data_platform#33 · created 1 week ago by yinglun.song@duke.edu Done	 updated 9 minutes ago
 Share with group team-hotel/pred_market_data_platform#34 · created 1 week ago by yinglun.song@duke.edu Current in Progress	 updated 8 minutes ago
 Deployment (Gitlab -> virtual machine) team-hotel/pred_market_data_platform#35 · created 1 week ago by yinglun.song@duke.edu	 updated 7 minutes ago
 Automatically starting team-hotel/pred_market_data_platform#36 · created 1 week ago by yinglun.song@duke.edu	 updated 7 minutes ago
 Security Analysis - DFD threat model team-hotel/pred_market_data_platform#37 · created 1 week ago by yinglun.song@duke.edu Current in Progress	 updated 8 minutes ago
 1st big picture draft team-hotel/pred_market_data_platform#38 · created 1 week ago by yinglun.song@duke.edu Done	 updated 8 minutes ago
 Refinement and detailing team-hotel/pred_market_data_platform#39 · created 1 week ago by yinglun.song@duke.edu Current in Progress	 updated 7 minutes ago
 Users and Administrative Functionality team-hotel/pred_market_data_platform#40 · created 1 week ago by yinglun.song@duke.edu Current in Progress	 updated 7 minutes ago

*Some of the issues that are created

*Updated for Sprint3

Updated Design

Architecture

Data source: Polymarket.

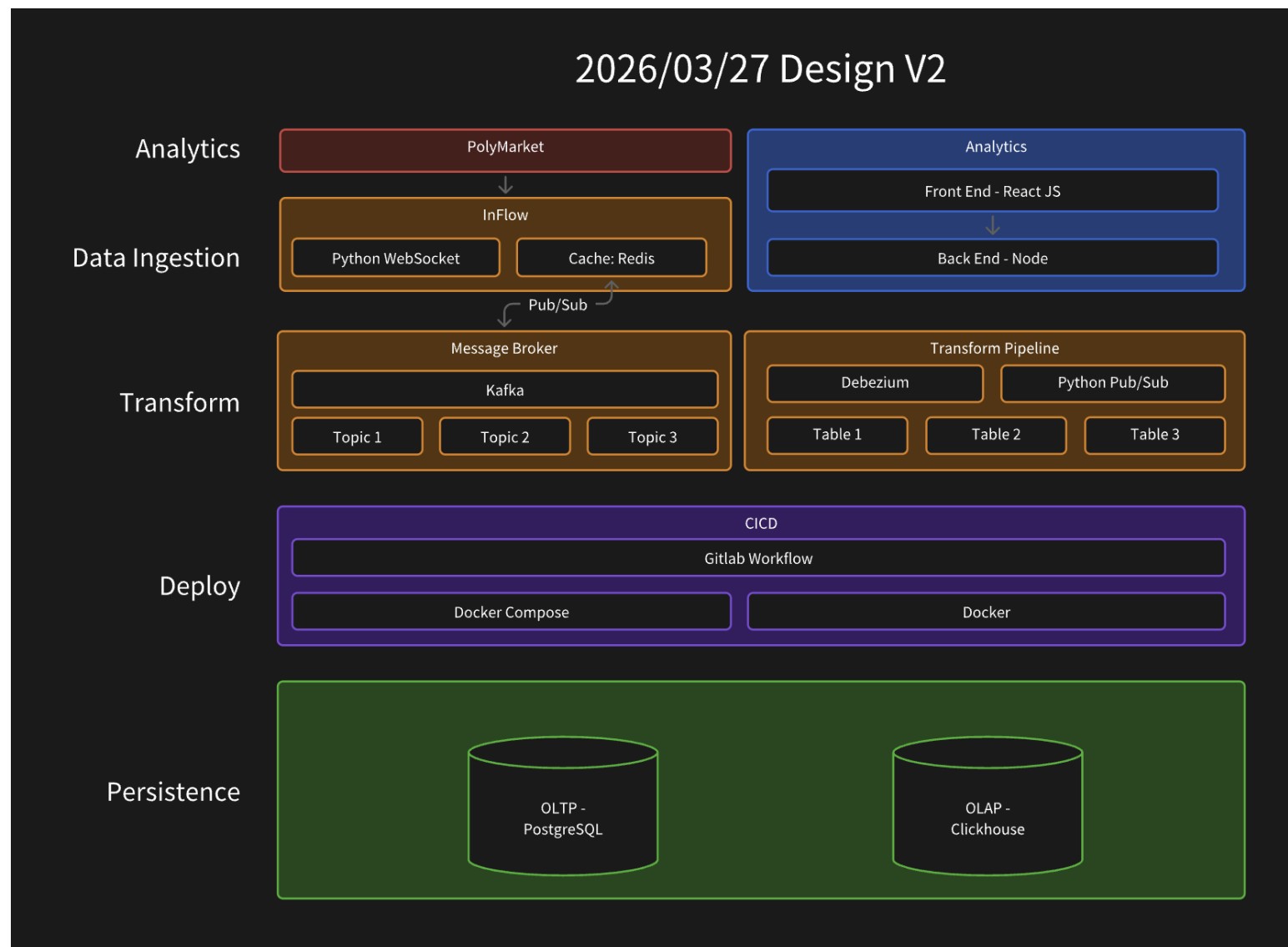
Ingestion layer: Python WebSocket collector with Redis cache.

Streaming backbone: Kafka message broker with multiple topics.

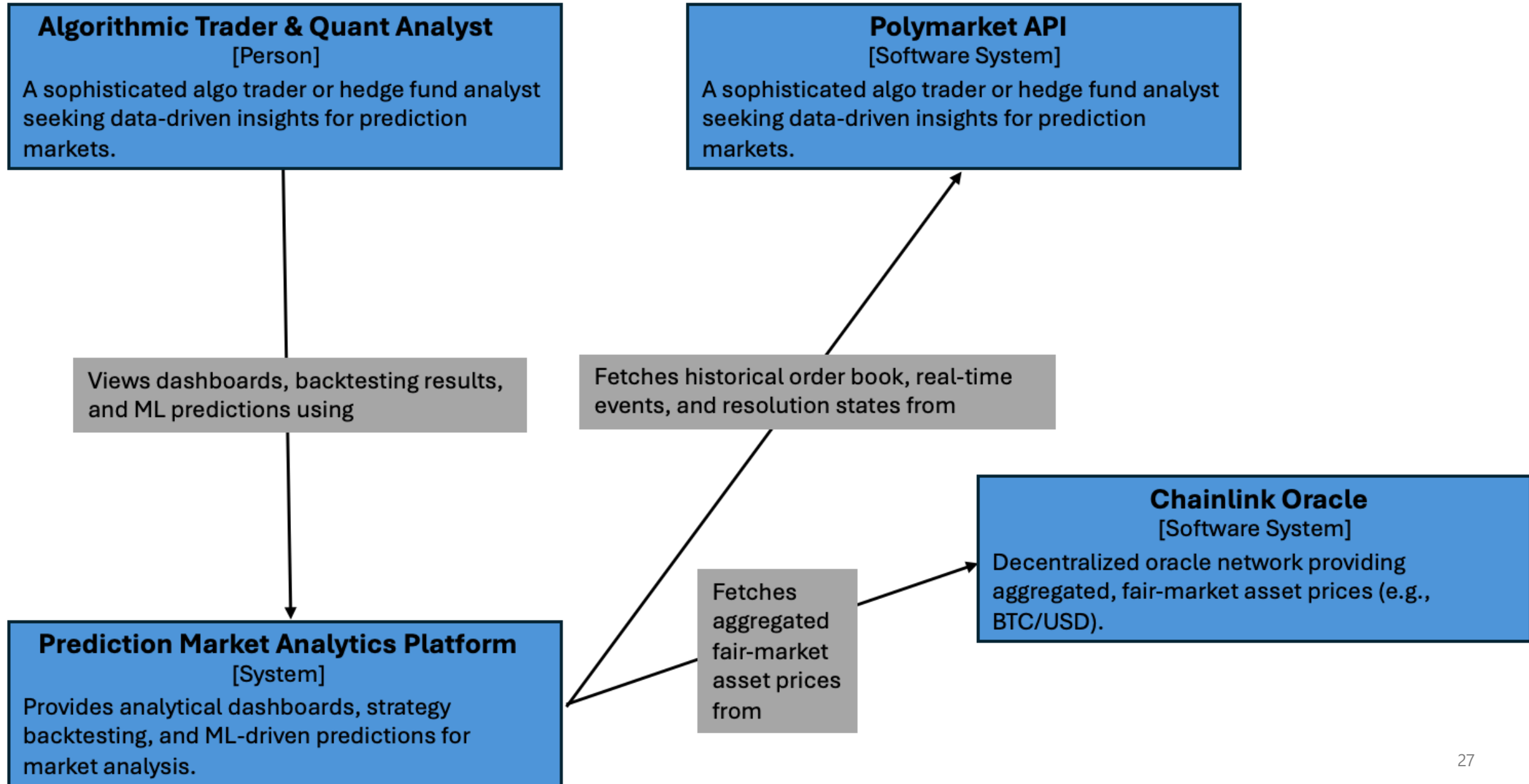
Storage: PostgreSQL (OLTP) for persisted normalized records.

Analytics backend: NodeJS services querying and aggregating stored data.

Analytics frontend: React UI for dashboards and research access.



System Architecture (C4 model) - Level 1 Context

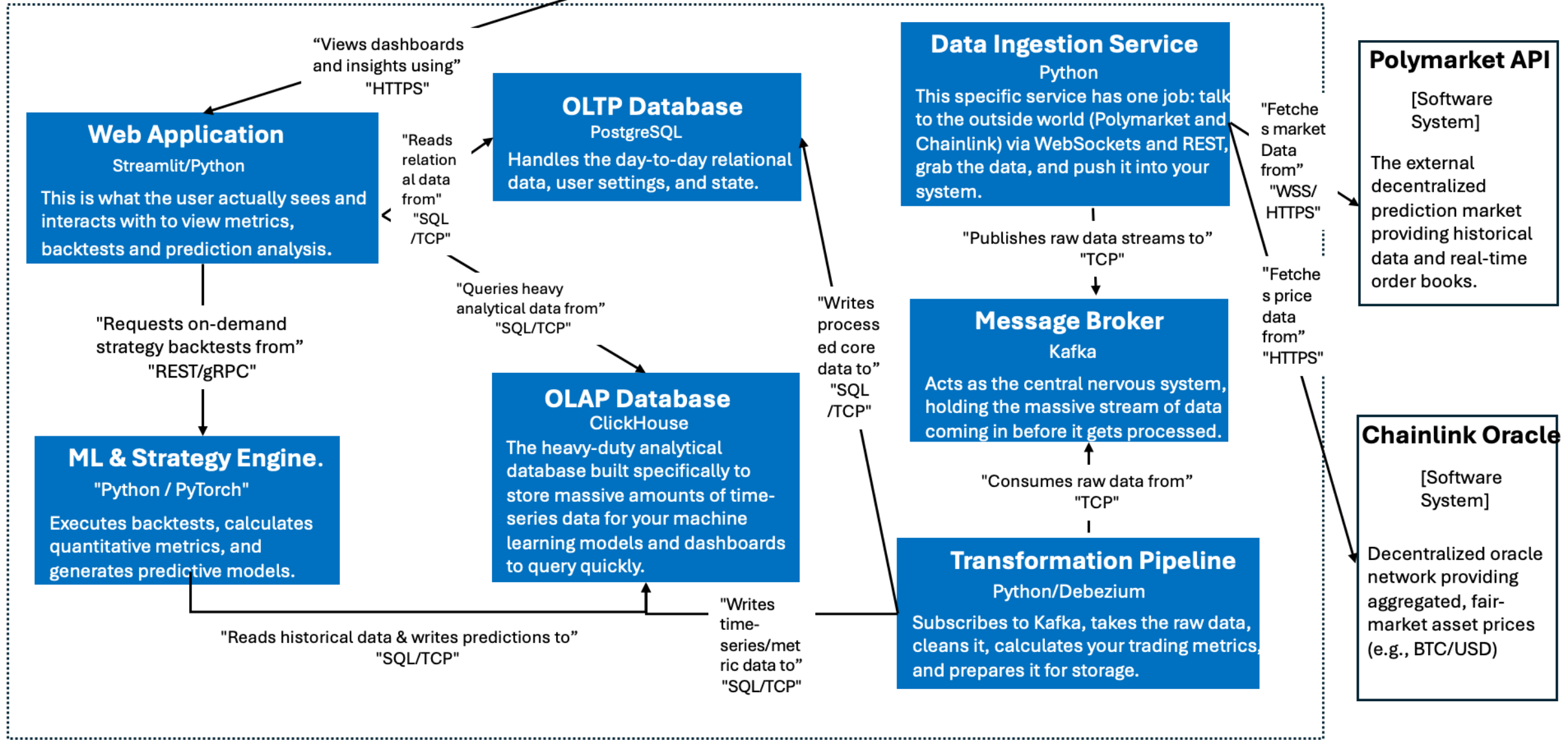


System Architecture (C4 model) - Level 2 Container

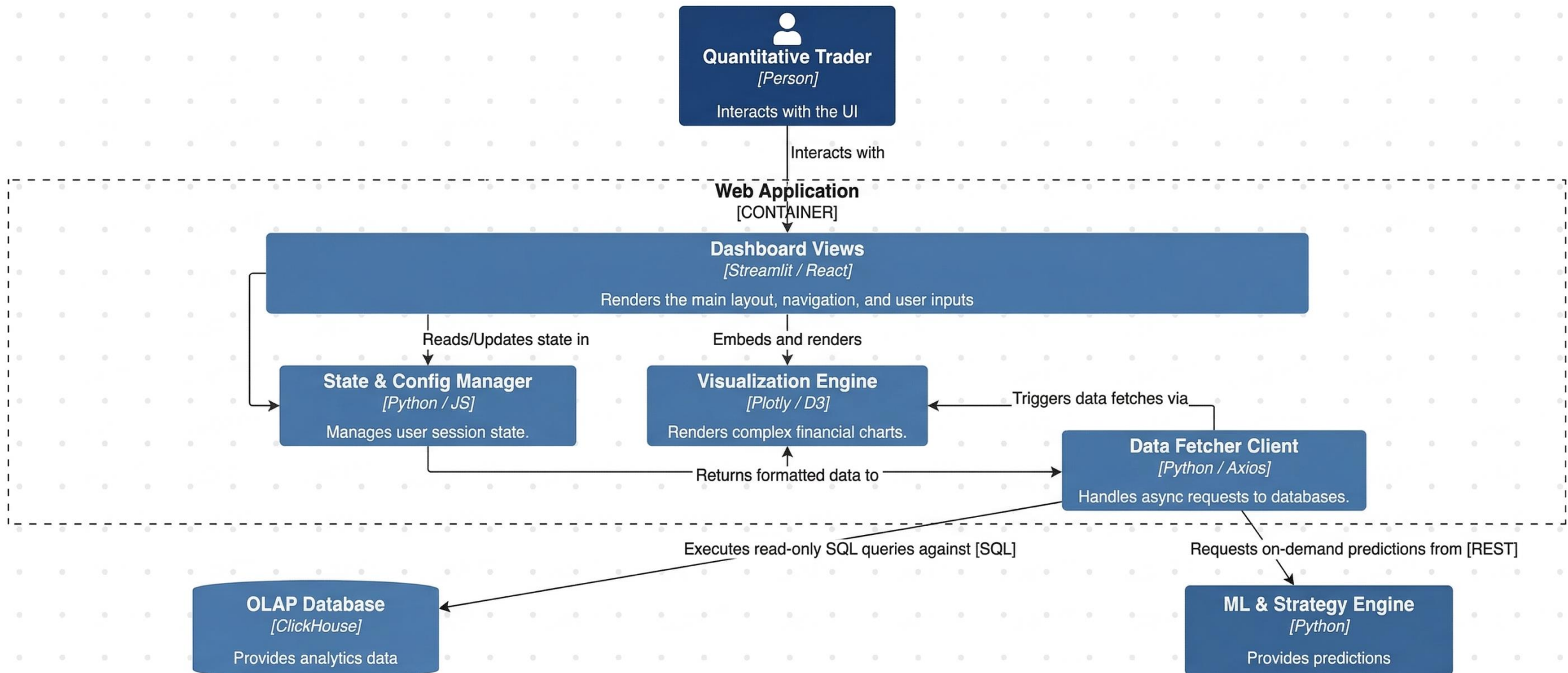
Algorithmic Trader & Quant Analyst

[Person]

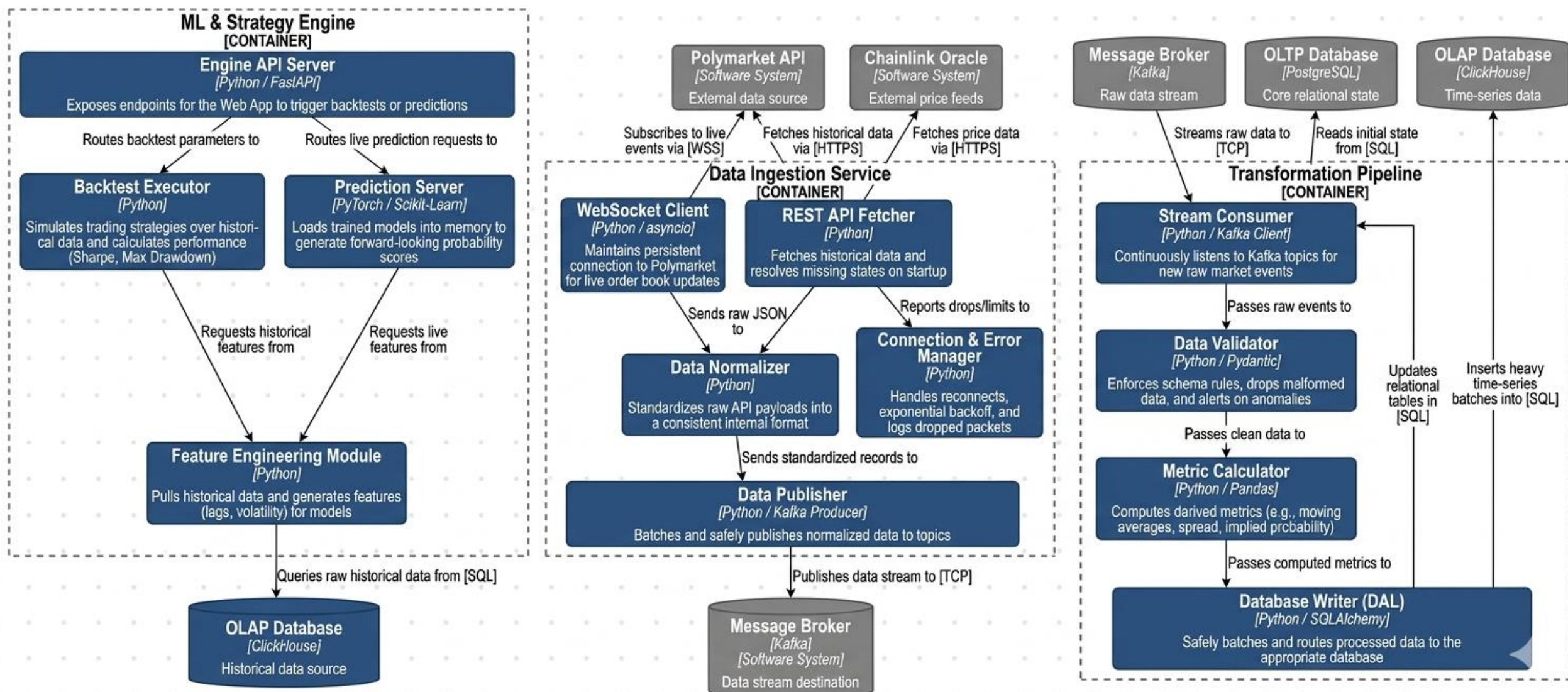
A sophisticated algo trader or hedge fund analyst seeking data-driven insights for prediction markets



Level 3 Component – Web Application



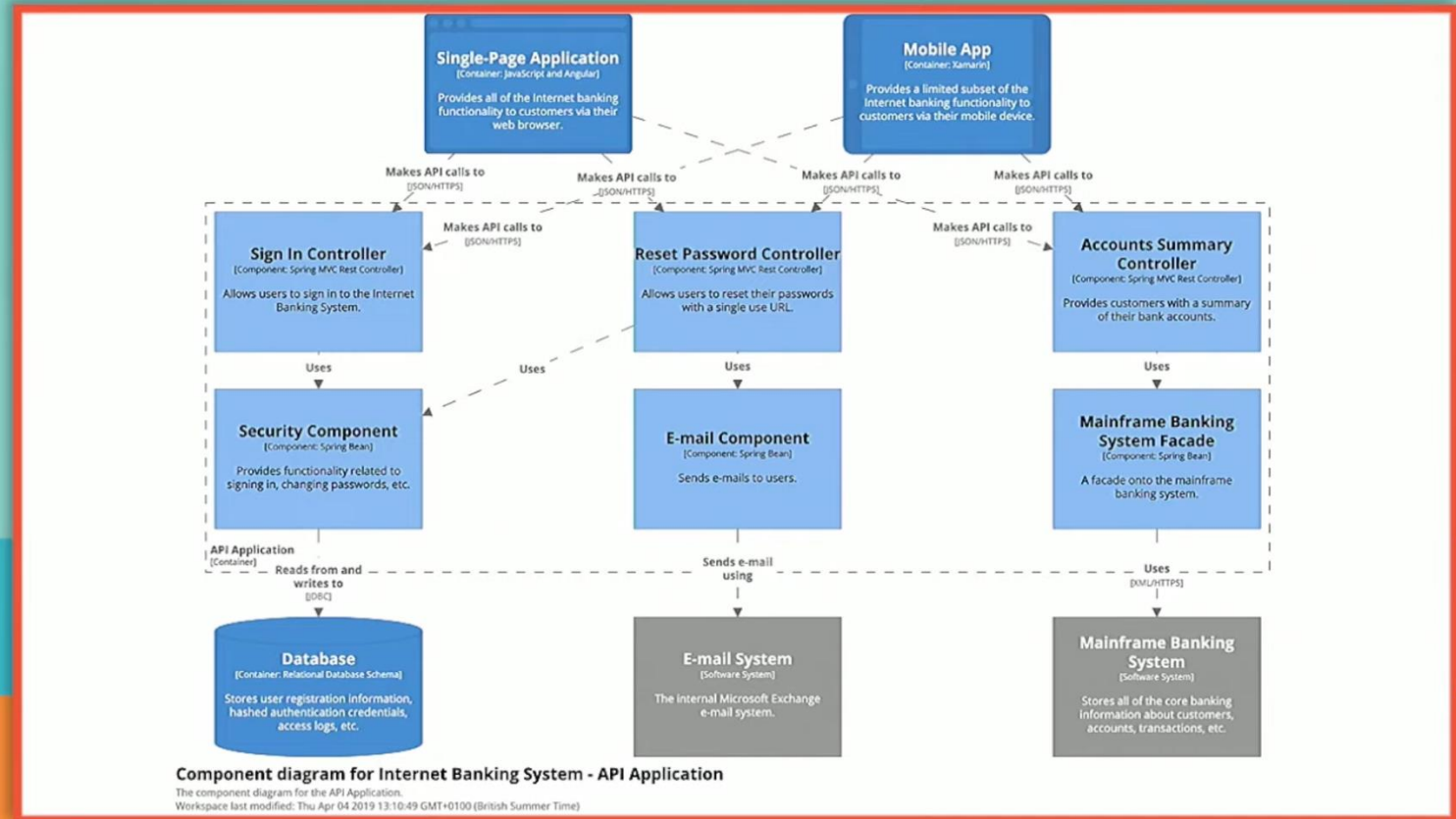
Level 3 Component – The Data Ingestion & Processing Containers



Level 4 Code



AGILE ON THE BEACH



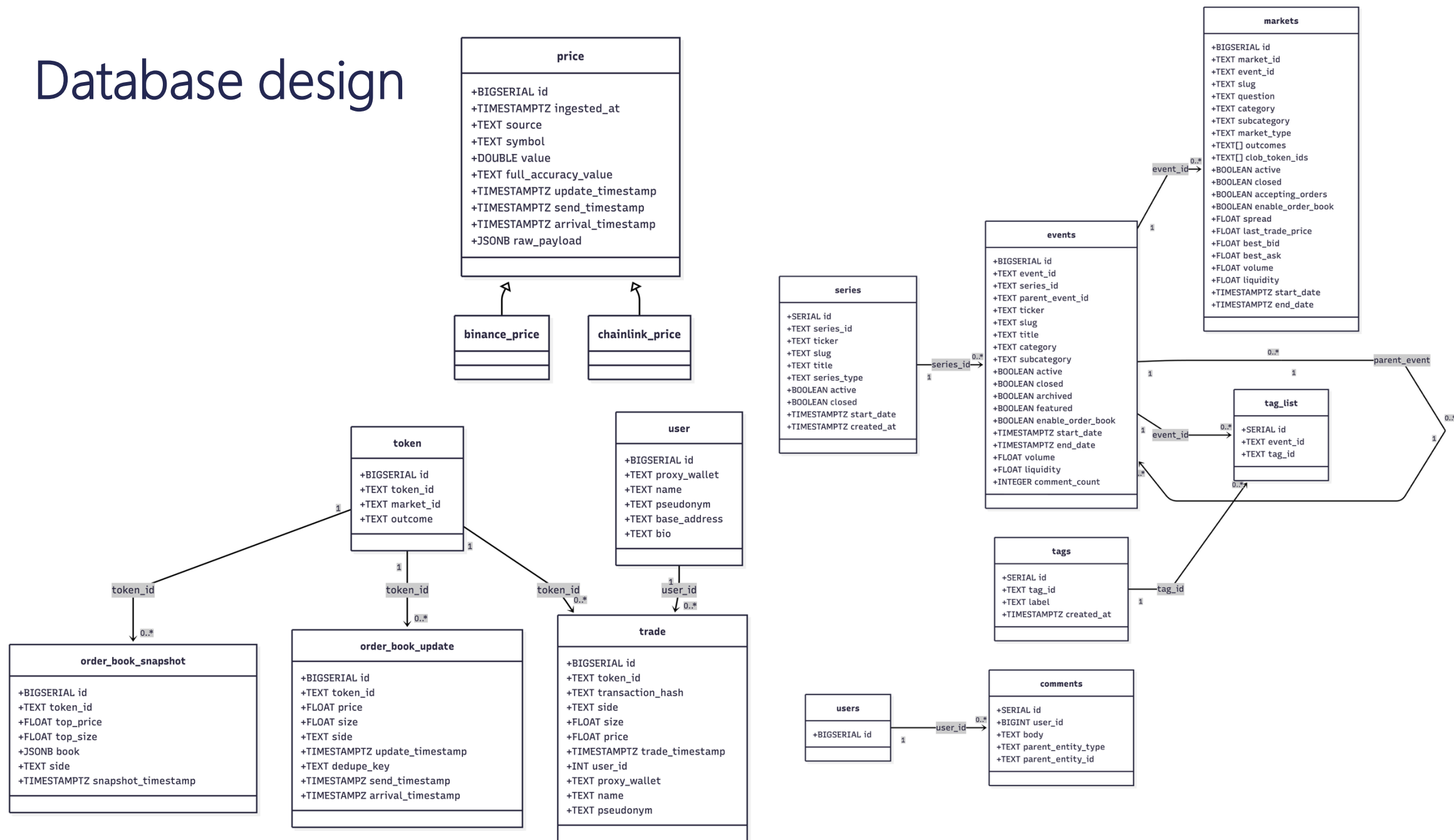
Streaming
sponsored by:

FALMOUTH
UNIVERSITY

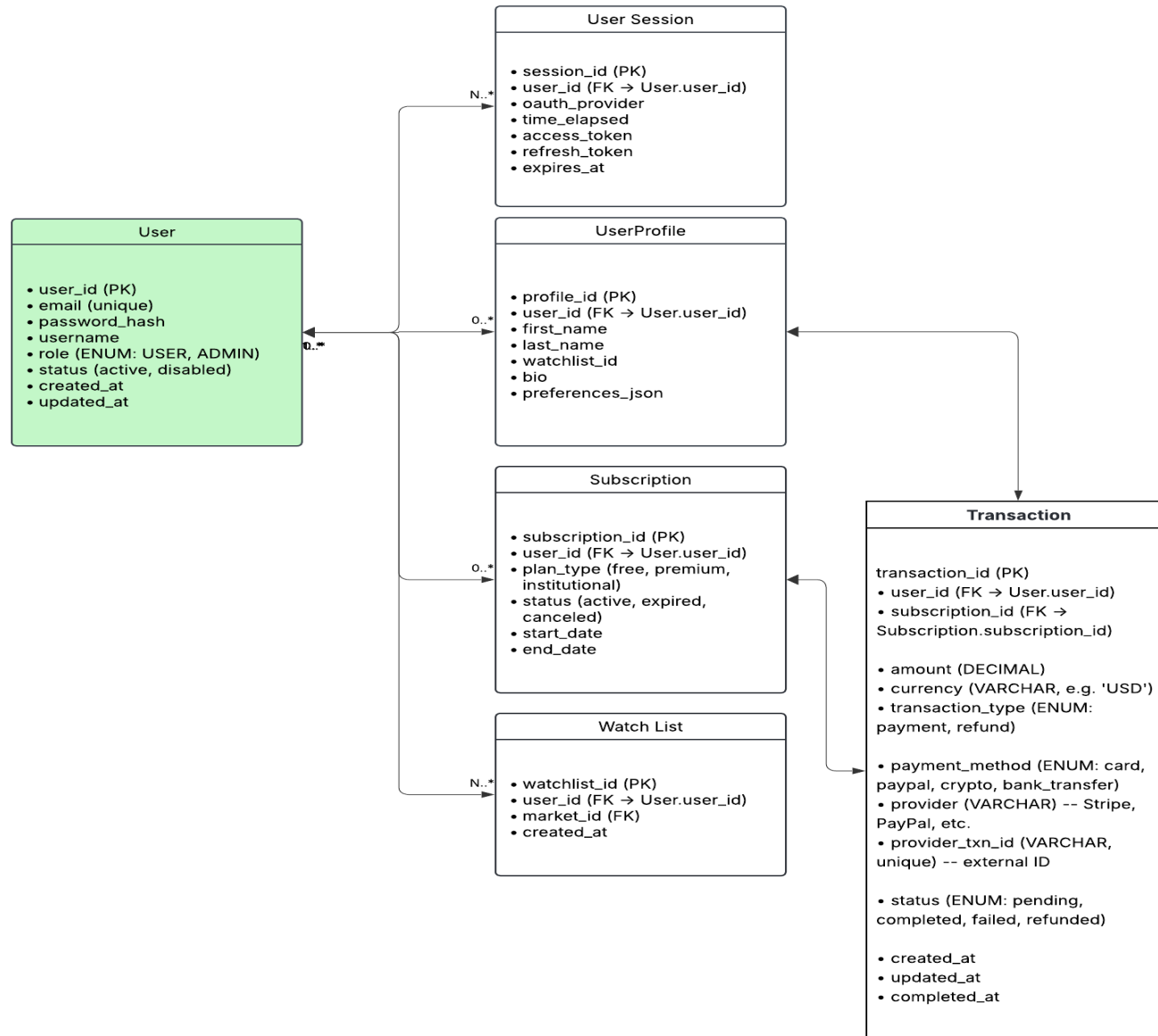
FLEXIBLE
LEARNING

always miss this I do not recommend
doing level 4 because it's not worth it

Database design



Database schema for users



Testing approach Methodology

Unit Tests (The Foundation):

Testing individual functions in isolation.

Task: Does this specific Python function correctly parse a Polymarket probability format into a float?

Integration Tests (The Middle):

Testing whether decoupled system modules communicate correctly.

Task: Does the WebSocket ingestion service correctly write data to the cache?
Does the message broker correctly ingest and forward the event?

Data Quality & Contract Tests (Pipeline Specific):

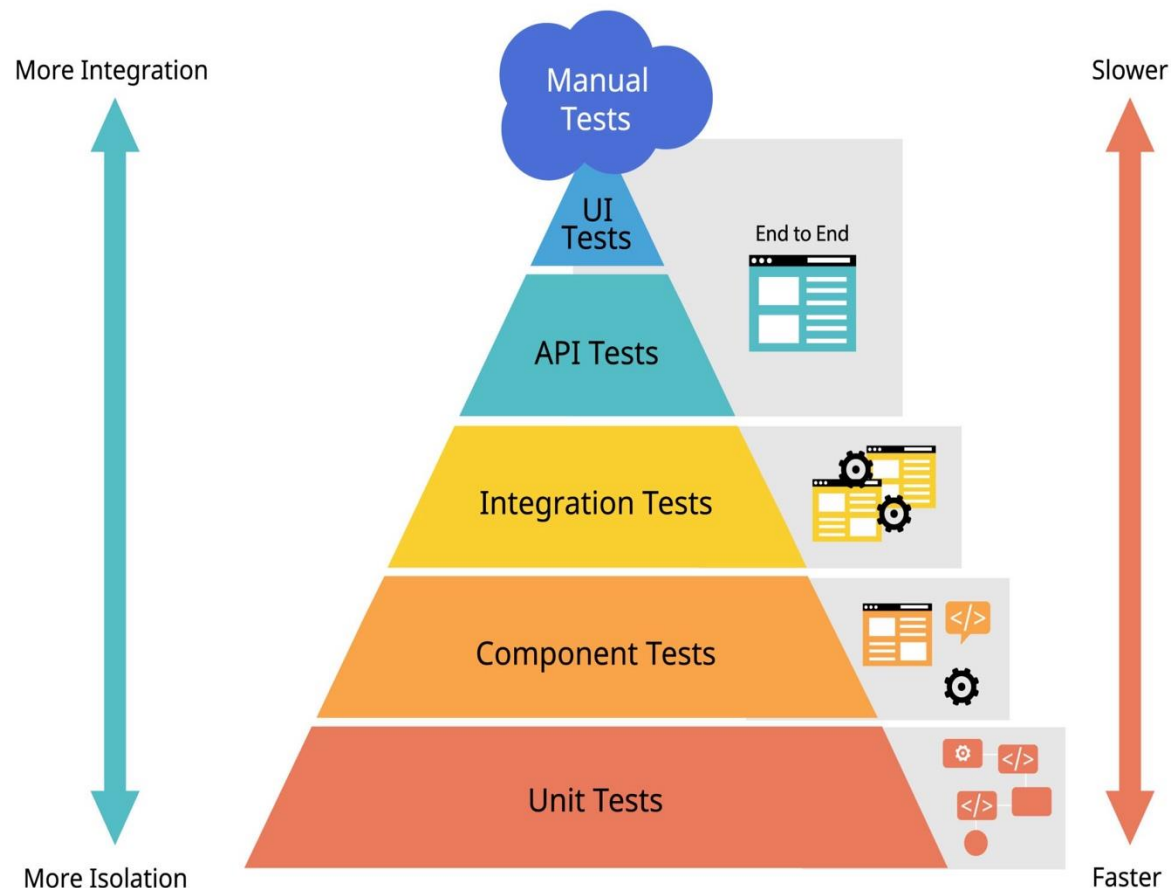
Testing whether schema or upstream data changes are handled safely.

Task: If Polymarket adds a new field to their JSON response, does our pipeline safely ignore it or crash?

End-to-End (E2E) Tests (The Peak):

Testing whether the full system works correctly from input to final user-visible output.

Task: If a trade event happens on the mock Polymarket WebSocket, does it successfully propagate through the pipeline, get stored in OLAP database, and appear correctly on the dashboard?



Testing approach In Data Ingestion

Corner Cases

Network Jitter & Disconnects.

What happens if the Polymarket WebSocket drops packets or silently disconnects? Does the system recognize the stale state?

Malformed Payloads.

Polymarket sends an unexpected null value for a betting pool size, or a string instead of an integer.

Redis Memory Eviction.

If the downstream pipeline slows down and Redis fills up, do we drop new data or block the WebSocket?

Examples from unit test to E2E test

Data Parsing

Objective: Verify that raw JSON from Polymarket is correctly transformed into our internal Python Data Class / Pydantic model.

Implementation: We do not need an internet connection for this. We save a sample Polymarket JSON response as a local fixture file. We write a pytest that feeds this static JSON into our parser function.

Python InFlow to Redis

Objective: Verify that parsed data is successfully cached in Redis with the correct Time-To-Live (TTL).

Implementation: We use Testcontainers (a framework that spins up a lightweight, throwaway Docker container of Redis just for the test). Our Python code connects to this temporary Redis instance, writes a mock message, and reads it back.

The Ingestion Slice

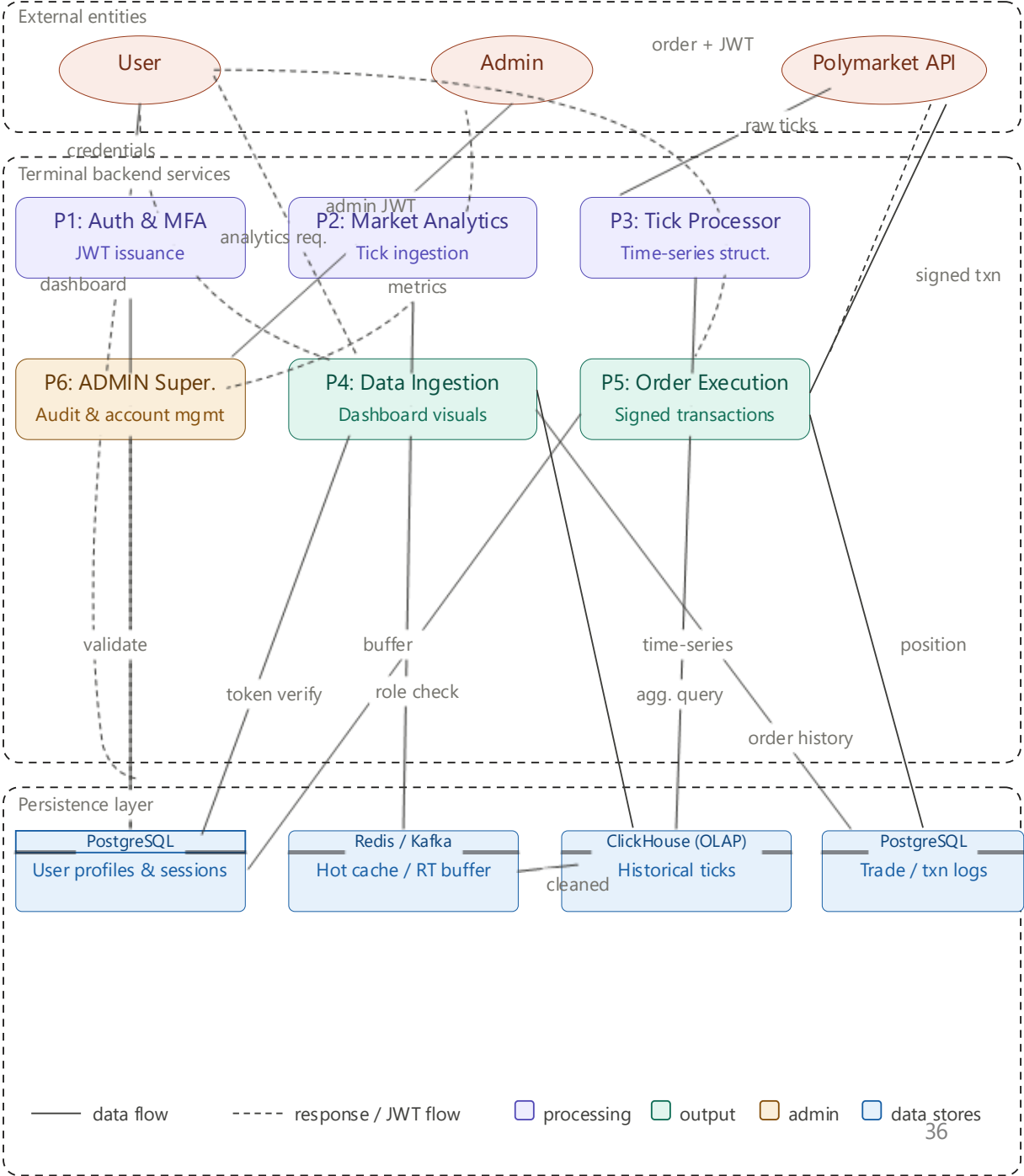
Objective: Verify the entire Inflow module works from WebSocket to Message Broker.

Implementation: We create a mock WebSocket server that mimics Polymarket and emits pre-determined trade events at a specific rate. We spin up our Python InFlow service, a Redis instance, and a single Kafka node using Docker Compose.

Data Flow Diagram (DFD)

System Overview:

- Users interact with the system through frontend connected to backend APIs
 - Backend services process authentication, logics and provide database access
 - External market data – Polymarket is ingested, processed and stored
 - Data views as dashboards and analytics components.
-
- **Assets:** User credentials and Oauth tokens, user permissions, Orderbook data and trade records, market structure data, internal APIs and service endpoints, PostgreSQL database contents, external market data (Polymarket based), system logs, audit records.
 - **Actors:** User, Administrator, Polymarket API, Backend Services (for handling logics and data processing)
 - **Threats:** Based on STRIDE analysis



Data Flow Diagram (DFD) – STRIDE analysis

All S T R I D E

CATEGORY	SEV.	NODES	THREAT	MITIGATION
Spoofing	H	User P1	JWT/session token stolen & replayed	Short-lived tokens + revocation list in D1
Spoofing	H	PoLyAPI P2	Polymarket API spoofed, fake tick data injected	TLS cert pinning + HMAC signature on every feed
Tampering	H	P5 PoLyAPI	Order request tampered before signing	User-signed payload; P5 verifies hash against receipt
Tampering	H	P2 D2	Tick data corrupted in Redis/Kafka buffer	Per-message checksum; P3 validates before writing D3
Repudiation	H	P5 D4	User denies placing order — no audit trail	Append-only signed order log in D4
Repudiation	M	P6 D1	Admin denies executing command	Immutable admin audit log; ship to off-system SIEM
Info disclosure	H	D1 P1	User PII leaked via errors or SQL injection	Parameterised queries; encrypt PII columns; no stack traces
Info disclosure	H	P6 PoLyAPI	API keys exposed via overexposed admin endpoint	P6 behind VPN/allowlist; secrets manager for all keys
Denial of service	H	P2 D2 P3	Tick flood overwhelms data pipeline	Rate-limit at P2; back-pressure on Kafka; circuit breaker
Denial of service	M	P4 D3	Expensive analytics queries degrade OLAP store	Per-user query budget + result caching in P4
Elevation of priv.	H	P6 P4 P5	Regular user JWT accepted on admin P6 endpoints	Deny-by-default middleware; role validated on every request
Elevation of priv.	H	P5 D1	JWT role claim tampered to bypass order checks	RS256 signing; re-fetch authoritative role from D1

STRIDE analysis:

- Spoofing
 - Stolen session or OAuth token, external data source is not true.
- Tampering
 - Modified API requests, data corruption and wrong market data.
- Repudiation
 - Users/admin deny actions, missing logs for audit.
- Information disclosure:
 - Data leakage like user information and API keys, and overexposed admin endpoints.
- Denial of service
 - Data pipeline overload.
- Elevation of privilege:
 - Users access admins endpoints, and lack of authorization check.

Testing approach - Unit test suite for Full Stack

Corner Cases

Invalid User Inputs. (frontend validation)

A user submits invalid format of forms, registration info, or empty data. The system needs to prevent submission and provide clear validation feedback in the UI level.

Unauthorized Access. (security)

A user tries to access admin routes or attempt to get access to API keys. Access control should timely focus on across both frontend and backend (render and authorization).

API Failure, Timeout or Error Response.

The backend may return an error or slow responses during user requests. The frontend should handle those scenarios without crashing or exposing internal errors.

Invalid Request, Malformed Payload (backend validation)

Users' requests may contain missing fields or incorrect data types or values. The backend should validate the inputs and reject those request correctly.

Examples

Frontend Component Validation

Objective: Verify that the UI components correctly validate the user input.

Implementation: We render components (login/registration) using testing libraries, simulate possible user inputs (forms/types), and verify validation behavior (without real APIs); thus, the test remain isolated at the component level.

API Helper Function (Frontend)

Objective: Verify that the frontend API utilities construct and handle the requests correctly from backend without too much lagging

Implementation: We monitor backend responses without calling a live server, and test the formats of requests, success handling and error states. We then verify the successful responses or error messages are correctly shown.

Backend Authorization

Objective: Verify that the protected routes enforce role-based access controls.

Implementation: We simulate authentication tokens and user roles, then test if the endpoints returning the responses correctly. Such as 401 Unauthorized for missing or invalid authentication and 403 Forbidden for insufficient privileges.

Backend Logic with interaction of Database

Objective: Verify if the logic independently of the database, including validating request payloads, transforming input data, returning expected outputs and failure case handling.

Implementation: We call on the database and test if the controller or service functions ensures the correct outputs and the error handling. For example, we simulate correct reads/writes, missing records or database exceptions.

Testing approach - Unit test suite for Inflow Gateway

For Inflow Gateway, we use a strong mocking setup for unit tests.

- pytest monkeypatch
- custom test doubles
- fake WebSocket connections
- recording publishers
- fake HTTP responses

On the right are a few corner cases, and how we test for them.

1. External dependency failures

- WebSocket API unavailable
- Redis unavailable
- network I/O interruption

2. Data processing edge cases

- incorrect or unexpected data transformation
- invalid market filtering
- missing or incomplete payload enrichment

3. Resilience and recovery scenarios

- timeout during data fetch
- WebSocket connection failure
- retry behavior after failure
- graceful degradation when a dependency breaks

4. Data quality and integrity issues

- duplicate message detection
- malformed or partial payloads
- incorrect publish behavior

Testing approach - Unit test suite for Transform Layer

For Transform Layer, we also use a strong mocking setup for unit tests.

- pytest monkeypatch
- custom test doubles
- fake WebSocket connections
- recording publishers
- fake HTTP responses

On the right are a few corner cases, and how we test for them.

1. **Data processing edge cases:**

Non-JSON messages, non-dict payloads, invalid numeric fields, missing timestamps, incomplete snapshot payloads, and source or event-type routing mismatches.

2. **Resilience and recovery scenarios:**

Minimum poll interval enforcement, flush retry after insert failure, deferred retry for foreign-key violations, task shutdown and cleanup, and signal fallback behavior.

3. **Data quality and integrity issues:**

Duplicate-safe dedupe key generation, skipping malformed rows inside batched updates, preserving existing SQL LIMITs, appending missing LIMITs, and verifying correct topic/table routing for transformed records.

4. **External dependency failures:**

WebSocket API unavailable, Redis unavailable, Kafka producer not started, Postgres fetch failure, generic publish failure, and interrupted signal-handler registration.

APIs

External Polymarket Gamma REST endpoints documented:

- `GET /events`
- `GET /series`
- `GET /trades`

Internal persistence endpoint:

- `POST /internal/postgres/write`
- Allowed table targets:
- `chainlink\_prices`
- `binance\_prices`
- `kafka\_price\_changes`

Please refer to api.yaml file in the git repository for additional details.

Polymarket Gamma API 1.0.0 OAS 3.0

External API for Polymarket market data.

Servers

https://gamma-api.polymarket.com - REST API server

Filter by tag

default

GET /events Fetch market events

GET /series Fetch market series

GET /trades Fetch trade history

POST /internal/postgres/write Batch write to Postgres

Schemas

GammaEvent >

Error handling

Domain & Logic Safeguards (Task-Driven)

Malformed Ingestion Payload (Schema Changes)

Mechanism: Dead Letter Queue (DLQ). Catch `ValidationError`, route bad data to a separate Kafka topic, and skip to the next message without crashing the pipeline.

Quantitative Calculation Exceptions (e.g., Missing Data, `ZeroDivisionError`):

Mechanism: Graceful Degradation. Wrap core math in try-except blocks. Output `stale_data` flags or safe previous values to prevent NaN or Infinity from polluting the Clickhouse OLAP.

Infrastructure & System Safeguard (NFR-Driven)

Downstream Service Timeout (>500ms API/DB response)

Mechanism: Circuit Breaker Pattern. Temporarily halt requests to the failing service to prevent cascading failures; serve historical fallback data from the Redis cache.

Kafka/WebSocket Connection Drops

Mechanism: Exponential Backoff with Jitter. Catch `ConnectionError` and reconnect with increasing, randomized delays (e.g., 1s, 2s, 4s + random ms) to avoid network storming.

Spike in Trade Volume (Consumer Lag)

Mechanism: Backpressure & Auto-Scaling. Monitor Kafka consumer lag continuously. If thresholds are breached, trigger AWS ECS to dynamically spin up additional Python processing containers.

Non-Functional Requirements

Real-Time Performance

Low-latency ingestion and processing for live Polymarket data, enabling timely analytics and strategy response.

Reliability & Fault Tolerance

Automatic retry, reconnection, buffering, and graceful degradation to keep the pipeline running under partial failures.

Scalability

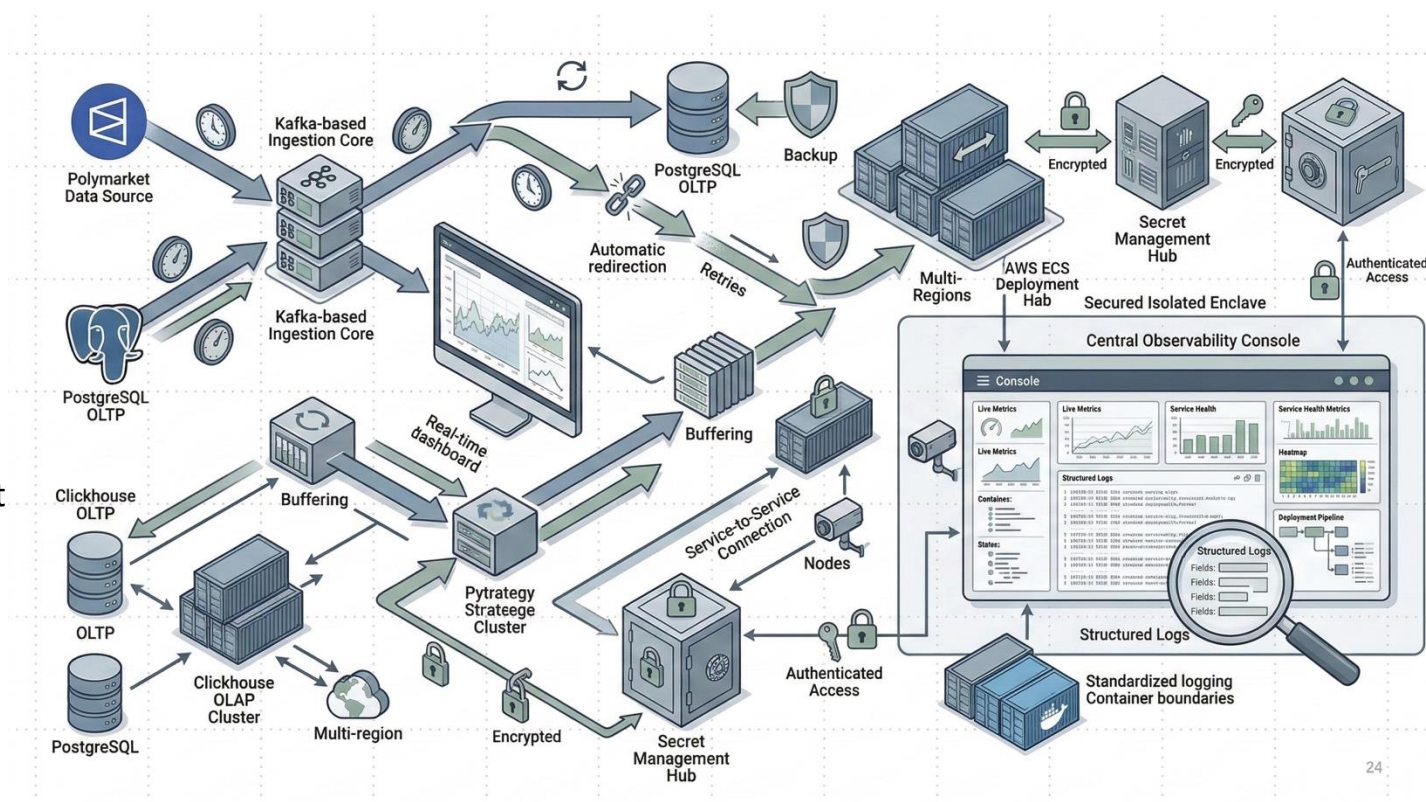
Modular services and message-driven architecture allow independent horizontal scaling as market volume grows.

Security & Access Control

Secure secret management, authenticated service access, and environment isolation to protect system resources and credentials.

Maintainability & Observability

Clear module boundaries, containerized deployment, structured logging, and monitoring to simplify debugging, testing, and future extension.

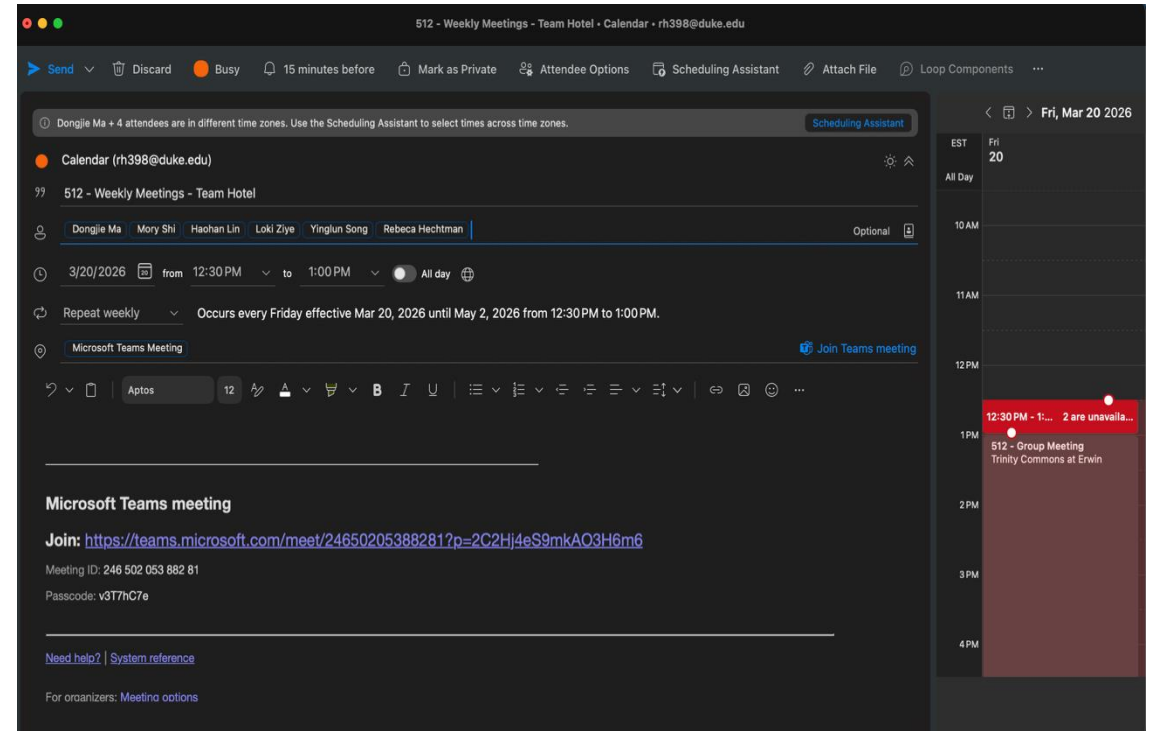
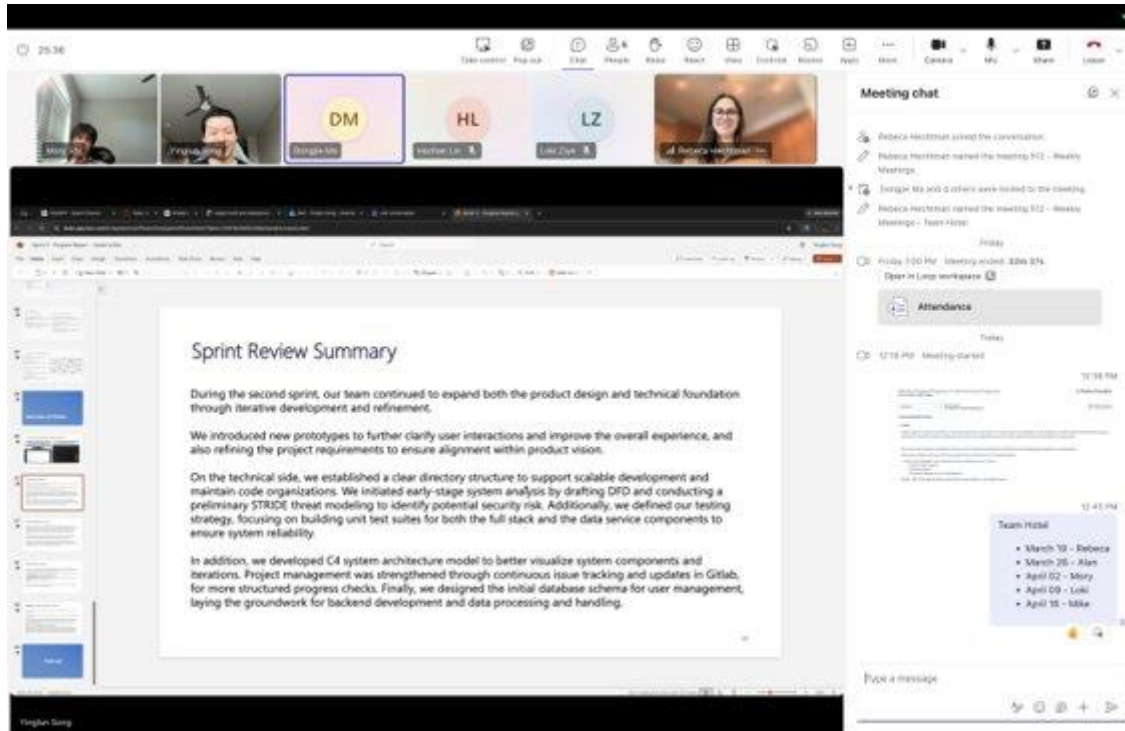


Status, Review and Planning

TA Discussion and Weekly Meetings Scheduled

We successfully met with our assigned TA today (March 27) to review our project progress and next steps

Next meeting with Dongjie Ma on April 03
(weekly meetings on Friday at 12:30)



Sprint Review Summary

Design & Prototyping: Building upon the finalized user interface prototypes, we have transitioned to developing initial code for advanced user features. Paper prototyping efforts continue to inform the detailed roadmap for charting and custom interactions.

Architecture & Analysis: A critical technical milestone was achieved with the refinement of the C4 system architecture diagrams and the detailed refinement and detailing of the Data Flow Diagram (DFD) and STRIDE threat model, which now provides a deep understanding of potential system risks.

Backend & Integration: Backend development has advanced significantly. The connection of the front-end to the API (JavaScript) is under development, and the PostgreSQL database schema for basic user management is established. Work to integrate the full stack with the data service is also in progress.

Processes & Collaboration: To streamline our development pipeline, the team conducted a targeted discussion on best practices for merge requests. We continue to utilize GitLab for continuous issue tracking and updates. Planning for subsequent tasks, including OAuth2 authentication and admin functionality, has begun.

Sprint Retrospective Summary

Looking back at Sprint 3, the team collaborated to put initial step in giving complex architectural decisions to small discussions, and have started building steps both in frontend and backend simulations. We had reached consensus in specific details in design and development progress as well.

Also, in technical side, we have separate the team into designing full stack and data, while also separating full stack into frontend and backend design, to make sure alignment we have did some discussion within it. And we have updated C4 and DFD in case more aligned with our final program. Additionally, the data is being prepared, as well as the final outcome expectations are being set.

For Sprint 4, we plan to continue our development progress, followed by the C4 model design principles. And we are setting goals and tasks status on GitLab to check process. We are planning to refine the test structures after we have initially compiled the program in full stack next week.

Sprint Planning Summary

For Sprint 3, our objective shifts from initial planning and strategy into active development and implementation. This process did not go without challenges:

1. Troubles with integrating the front end and back end design
2. API and stack coherent between full stack and the data layer
3. Deployment and share of the VCM

We were able to overcome these challenges and move forward with our project.

And for next week, sprint 4, we plan to tackle the highest-priority items currently in our project backlog.

Key deliverables for the upcoming week include

1. Initializing the user database,
2. Design of the Admin database
3. First version implementation of the full stack
4. Define the unit tests around core functions
5. Having the ETL pipeline up and running.
6. Share VCM with the team

We have scheduled our recurring weekly TA meeting for Friday 12:30 to ensure we remain aligned with the project constraints and technical requirements.

Additional Comments and/or Concerns

General Comments: It is almost the end of the project, we still have a lot to do. We still need to do a lot of coding. We are trying to make sure that every week we are doing what we need to do so that we can finish the project on time.

Risks/Blockers: Currently, there are no major blockers. However, we will be actively monitoring concerns as the ones below:

Risk1 – Operational Overhead: Our approach strongly resembles the microservice architecture, while it allows for better modulization, it also adds maintenance overhead to the team. We need to closely monitor the status of each service, and also added more automated orchestrations such as CI/CD flows to ensure smooth operations.

Risk2 – API Access Concern: Polymarket may retrieve/change the accessibility to their official API endpoints, and it will significantly impede our development.

Risk 3 – Prediction Model Performance Risk: The performance of the prediction model may not meet the required accuracy or reliability thresholds. If the model fails to generate meaningful predictions, the analytics component may not provide sufficient value for deployment in the production dashboard.

Risk 4 – Massive Data Storage Need: As we are capturing high frequency tick-level orderbook data, the need for storage is overwhelming. We are looking at the need of roughly ~20 GB of storage need per day.

Annex (initial documents created)

Link to your project requirements document

<https://duke.box.com/s/b9j6rdc5zg165nhuga59pv1w49vvsqz5>

Major Requirements:

1. Main Functionality Requirements:

- Enable customers to interact with the website. Specifically, allow customers to create accounts, view data and analytics, and subscribe to the website.
- Maintain running instance of the website, running on a virtual machine to continuously pull data from polymarket and display on the website.
- Enable customers to check their account history and activities.
- Enable users to create new account for themselves. Tests available for developers of the team and TAs

2. Users Requirements:

- Add use cases for new user to access the website, subscribe to the website, view the data, charts, analytics, and check their account.
- Write test cases for checking 2 different types of historical data. e.g.
 - Check bitcoin up market data at 15 minutes duration, by tick, timestamp, December 9, 10:00AM
 - Check bitcoin down market data at 15 minutes, by 1 second buckets, timestamp, December 12, 10:00AM
- Parameters:
 - Market ID: Series number for different types of market
 - Book playback granularity: Duration of the buckets of displayed trades
 - Timestamp: the period of the data which the user wants to investigate.

3. Data Integration Requirements:

- The system shall integrate external data from at least one third-party REST/JSON API (e.g. Polymarket).
- Retrieve data programmatically via HTTP requests.
- Parse and validate JSON responses.
- Persist relevant data in the system database.
- Handle API errors, rate limits, and invalid responses gracefully.

4. Charting Requirements:

- Include the strike price, resolve price, and price at selection
- Include chain-link BTC/USD over time price chart
- Include Best Bid, Best Ask, Spread data
- Include a Mid Price (Snapshot view) over time chart
- Include a size over price chart
- Include cumulative depth over price chart
- Include the trader summary history
- Include a section of loaded data

5. UI/UX Design Focus - Prototyping I: Paper Prototype (Home page/Main page)

- Develop a Home page prototype that tells user what this website is about and display our prop.
- Develop a general data page prototype which will be where the main data charts reside. This will be the core function of this website. It will include all the charts mentioned above, allowing users to pick parameters of their desire and print the corresponding data and chart on the screen with already-calculated analytical indicators.

6. UI/UX Design Focus - Prototyping II: Raw webpages

- Fill in the design of other pages based on the style and core implementation of the main and home pages
- Come up with the overall design scheme, such as background color, font, icon, name, and viewports.

7. UI/UX Design Focus - Prototyping III: Final Polish

- Gather feedback from other peers and see what to improve.
- Make sure the overall style is coherent and consistent
- Make sure it is easy to use, and nothing is hard to see or find.

Project Specific Requirements:

1. System Architecture

- Data Ingestion (Hot Path):
 - Python WebSocket client to ingest market data (PolyMarket).
 - Redis cache for real-time data access.
 - REST/WS strategy execution service for trade decisions.
- Transform Layer
 - Kafka message broker with multiple topics.
 - Debezium for CDC from OLTP database.
 - Python Pub/Sub services for processing and routing events.
- Analytics & Deploy
 - Strategy and backtesting services (Jupyter, Streamlit).
 - Object-oriented strategy design.
 - Containerized deployment (Docker, Docker Compose).
 - CI/CD integration with cloud container registry.
- Persistence (Off Path)
 - OLTP database: PostgreSQL.
 - OLAP database: ClickHouse.

2. Implementation of CI/CD pipeline

- Builds the application on every commit.
- Runs the full automated test suite.
- Executes GitLab SAST scanning.

- Deploys the application to the target VM/container environment upon successful build.

3. Security Requirements

- Threat Modeling:
 - Create a Data Flow Diagram (DFD).
 - Identify assets, actors, and trust boundaries.
 - Perform STRIDE analysis on the system
- Static Analysis
 - Run semgrep on the codebase.
 - Address or document identified vulnerabilities.

4. Architecture Documentation (C4 Model)

- System Context Diagram
- Container Diagram
- Component Diagram
- Code-Level Diagram

5. Logging & Monitoring

- Follow OWASP Logging Cheat Sheet guidelines.
- Define:
 - Events to log
 - Log storage location
 - Log format (structured logging required)
 - Sensitive data handling
- Document in `log_design.md`
- Log security-relevant events (auth, failures, data access, trade execution).
- Ensure logs support incident investigation.
- Prevent log tampering and unauthorized access.

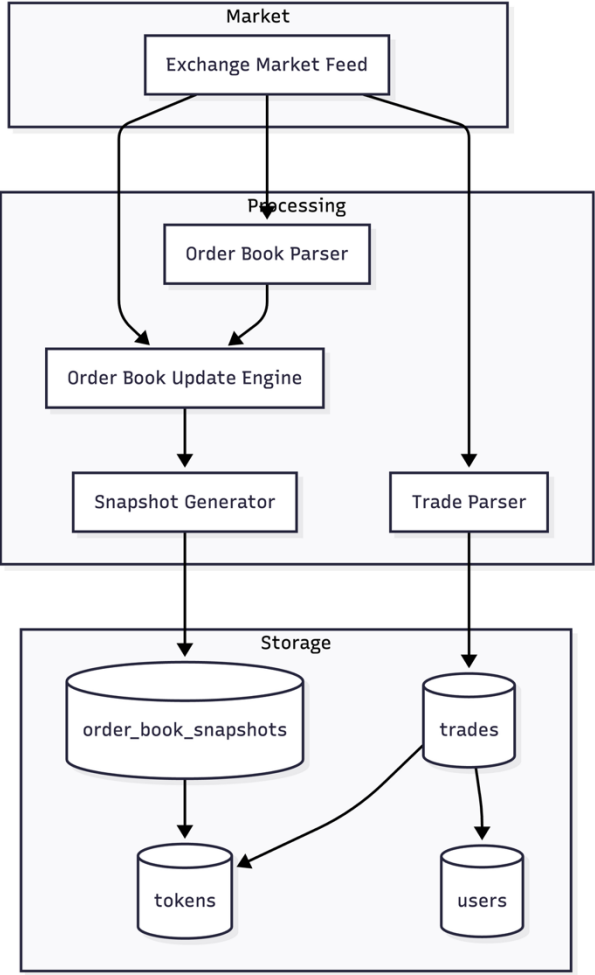
6. OAuth2 Integration Requirements

- Integrate a third-party OAuth2 provider (e.g., Google).
- Allow users to authenticate via OAuth2 login.
- Securely store and manage access tokens.
- Protect authenticated routes and restrict access to authorized users only.
- Ensure no user passwords are stored locally.

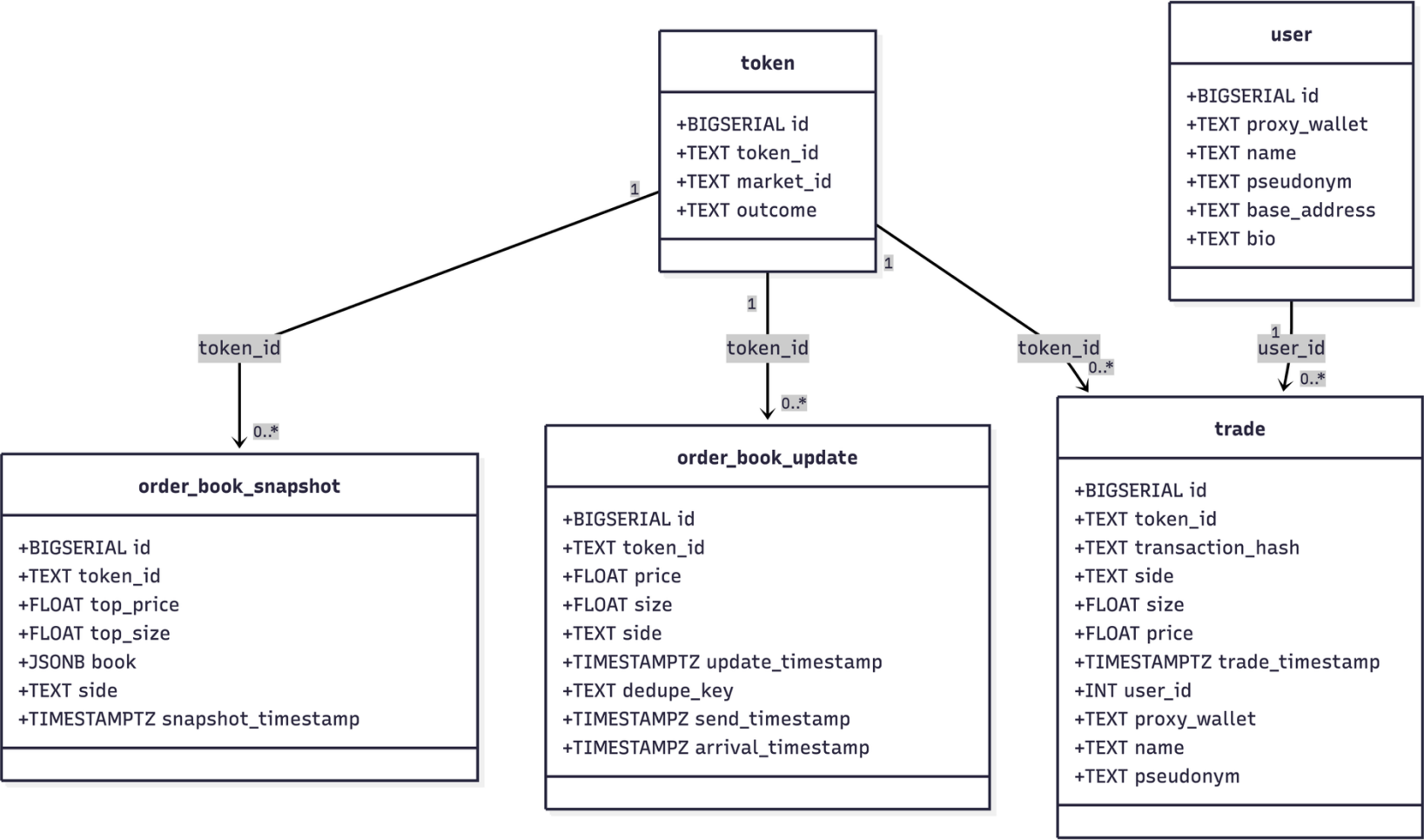
7. Containerization Requirements

- Provide a Dockerfile that builds a production-ready image of the BigBucks application.
- Provide a docker-compose.yml file that:
 - Deploys the application and required services (e.g., database).
 - Creates persistent volumes for database storage.
- Use Docker Compose to deploy and run the application on the target virtual machine.

UML Class Diagram – Order Book

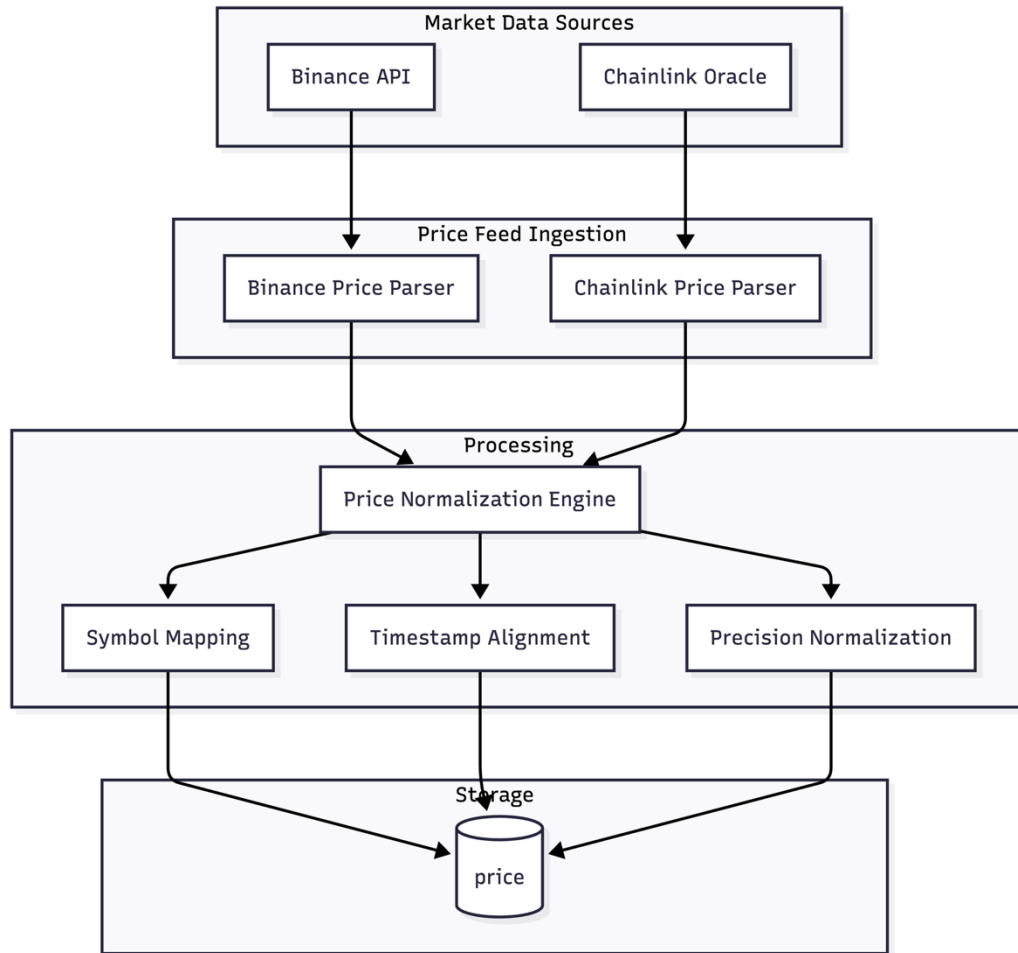


Architecture/pipeline

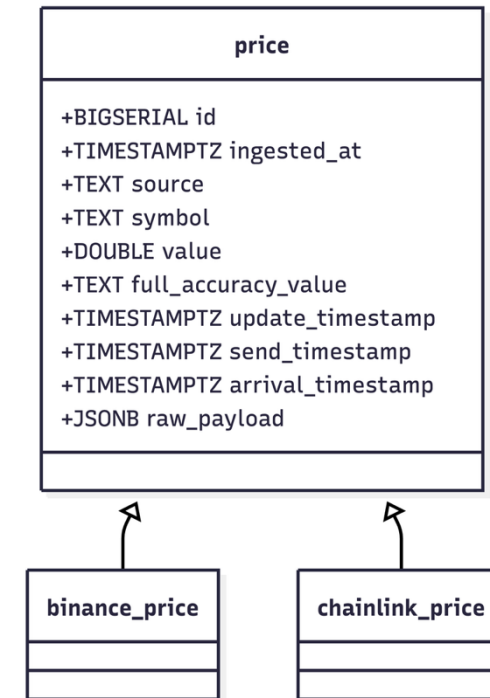


UML Class Diagram

UML Class Diagram - Price

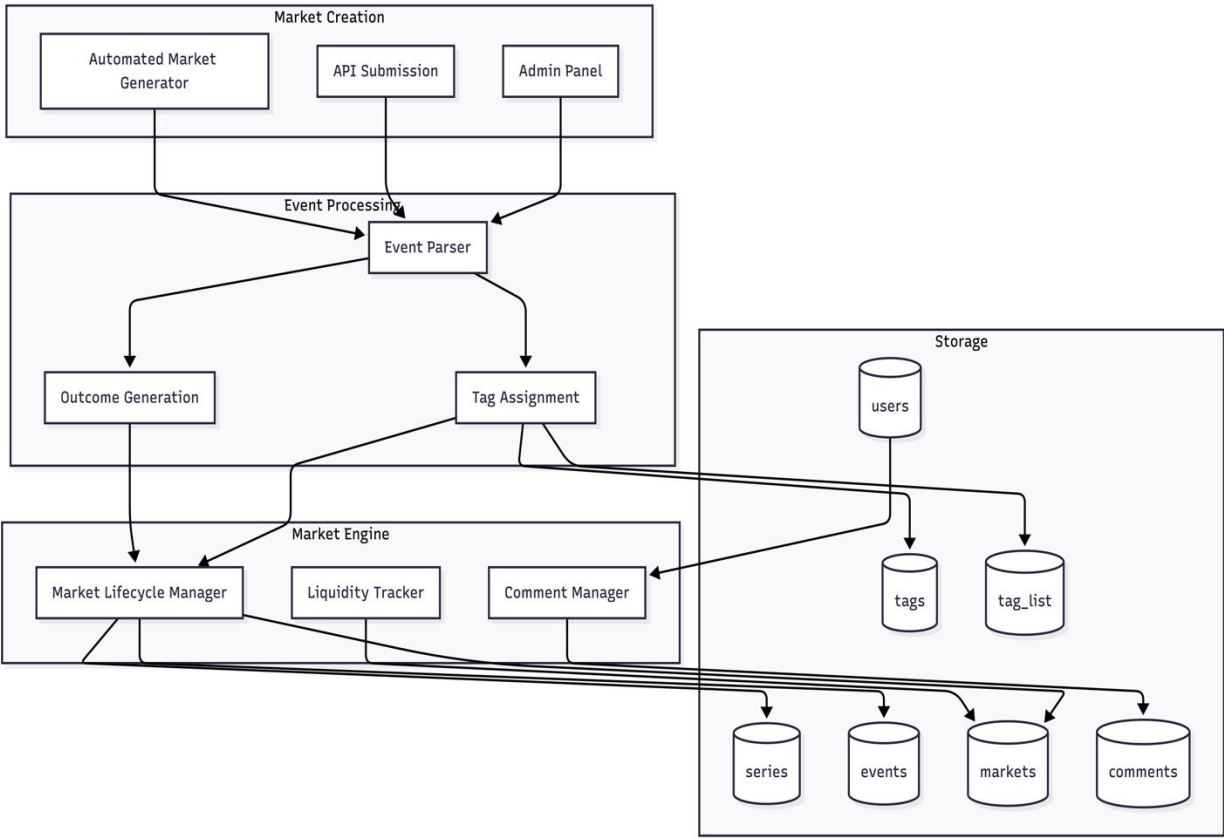


Architecture/pipeline

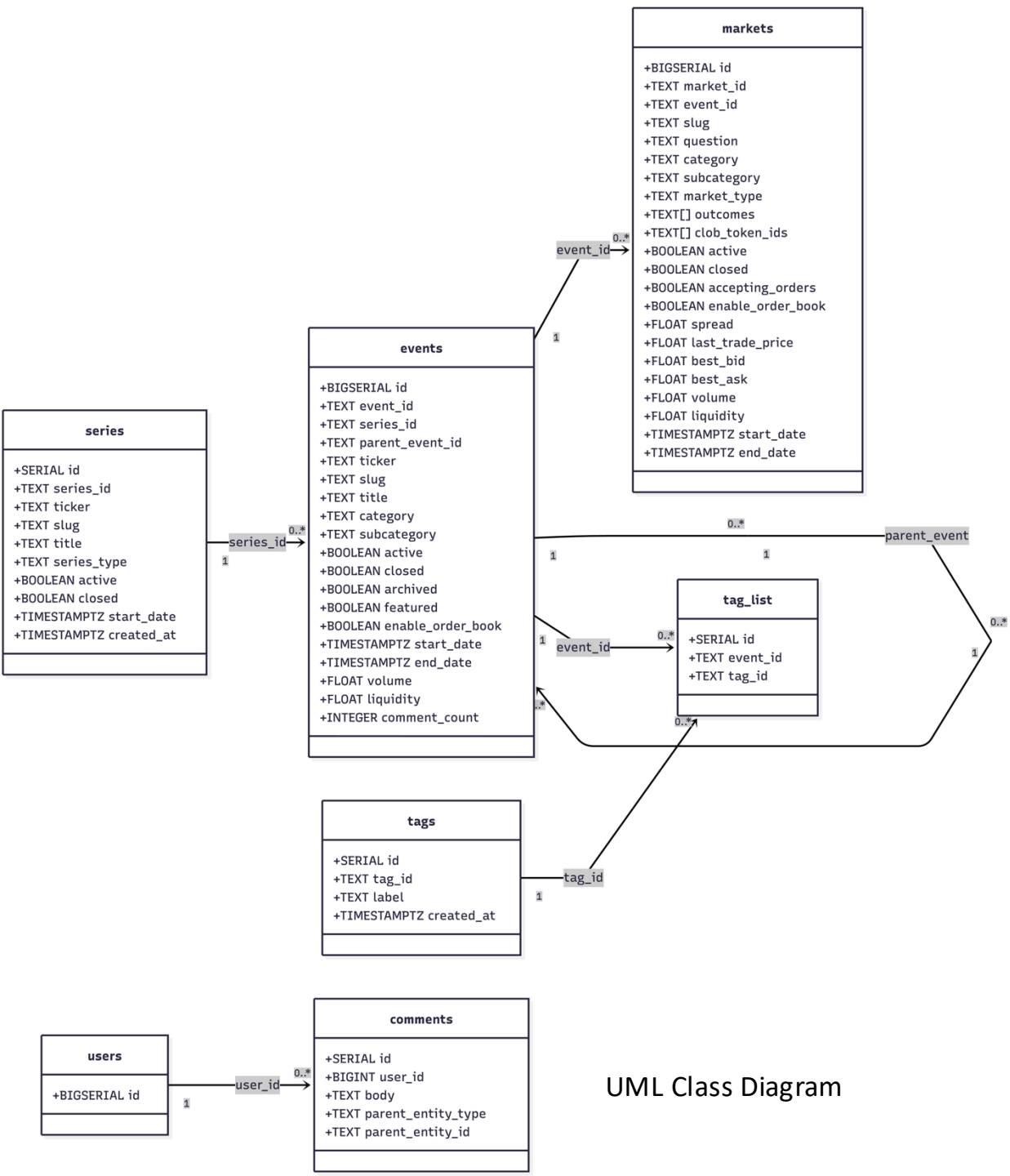


UML Class Diagram

UML Class Diagram - Market



Architecture/pipeline



UML Class Diagram

Class design

Trading/Order Book System

- Core class: token, order_book_snapshot, order_book_update, trade, user
- Relations:
 - Token -> order_book_snapshot
 - Token -> order_book_update
 - Token -> trade
 - User -> trade

Price Oracle System

- Core class: price, binance_price, chainlink_price
- Relations: price --> binance_price / chainlink_price

Market Structure System

- Core class: series, events, markets, tags, tag_list, comments, users
- Relations:
 - series -> events -> markets
 - events -> events (parent_event)
 - events -> tag_list -> tags
 - users -> comments

Usage

Store the order book state and trade executions for each token.

Provides external reference prices from different oracle sources.

Defines the hierarchical structure of prediction markets and their metadata.

Thank you!